

Архитектура Seq2Seq, механизм Attention, Transformer

План раздела

- Задача машинного перевода
- Архитектура Seq2Seq для решения задачи перевода и ее обучение
- Недостатки RNN-модели для решения задачи перевода

Задача машинного перевода

Эволюция подходов к решению задачи машинного перевода:

- Правильный МТ (1950-е)
- Phrase-based/статистический МТ (1990-е)
- МТ на основе нейросетей (2010-е)

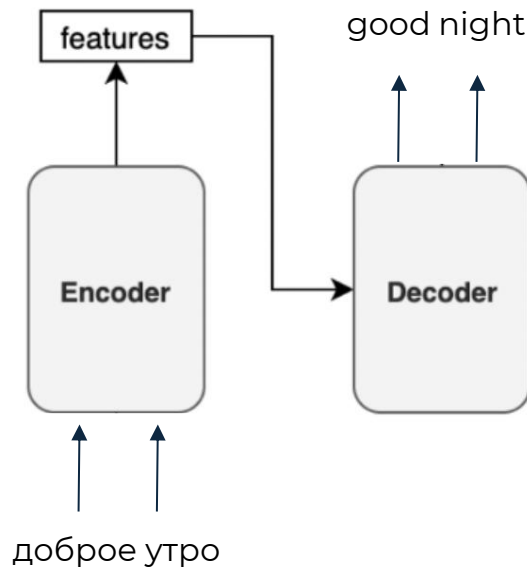
Хорошая и интересная статья об истории машинного перевода: [link](#)

RNN для машинного перевода

Идея модели:

Нейросеть будет состоять из двух частей: Encoder и Decoder.

- **Encoder** будет принимать на вход предложение на source языке и агрегировать информацию из него;
- **Decoder** будет генерировать предложение-перевод token за токеном на основе информации, которую передаст ему encoder.

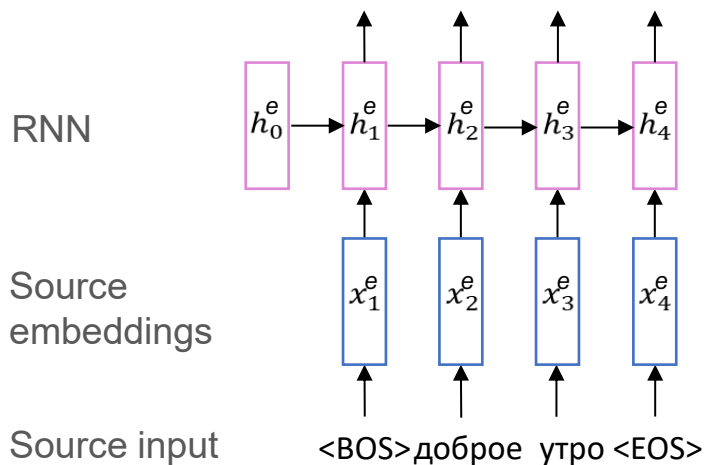


RNN для машинного перевода

Encoder

Энкодер состоит из слоя embedding и RNN слоев.

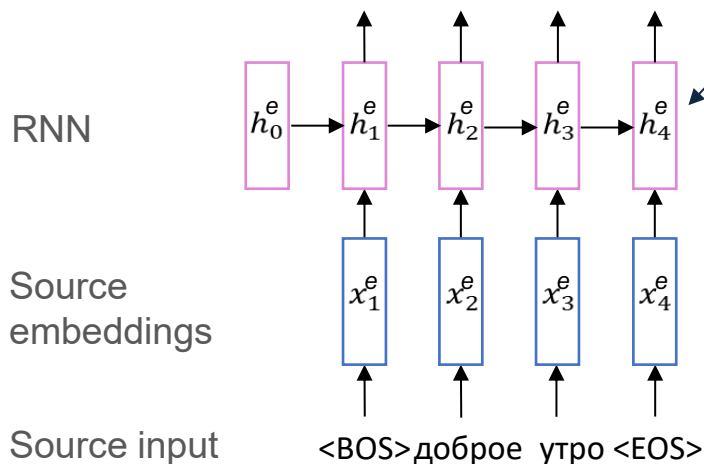
Он “читает” входную последовательность токен за токеном и обновляет скрытые состояния своих RNN-слоев

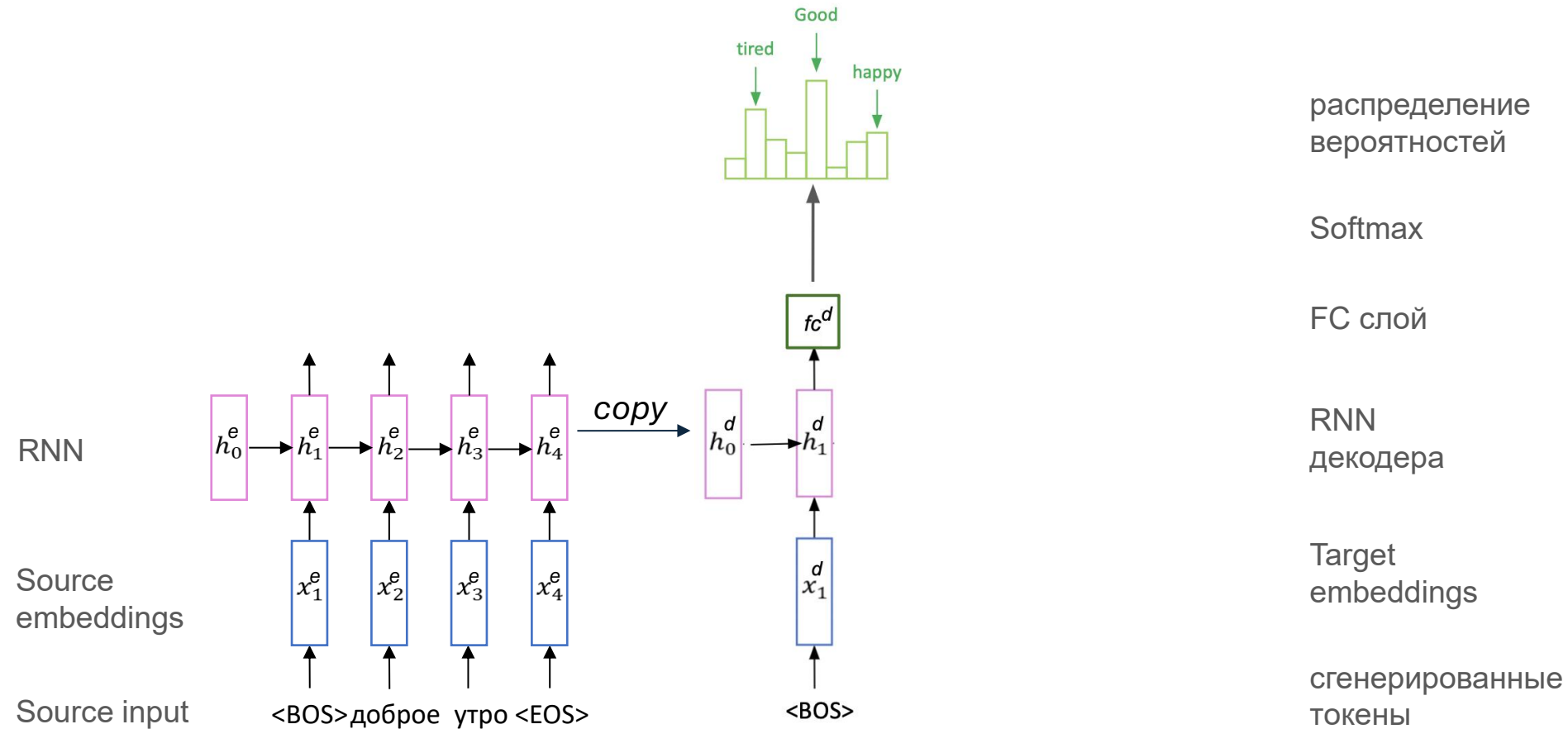


RNN для машинного перевода

Encoder

Скрытое состояние RNN в конце обработки предложения будет содержать агрегированную информацию о предложении

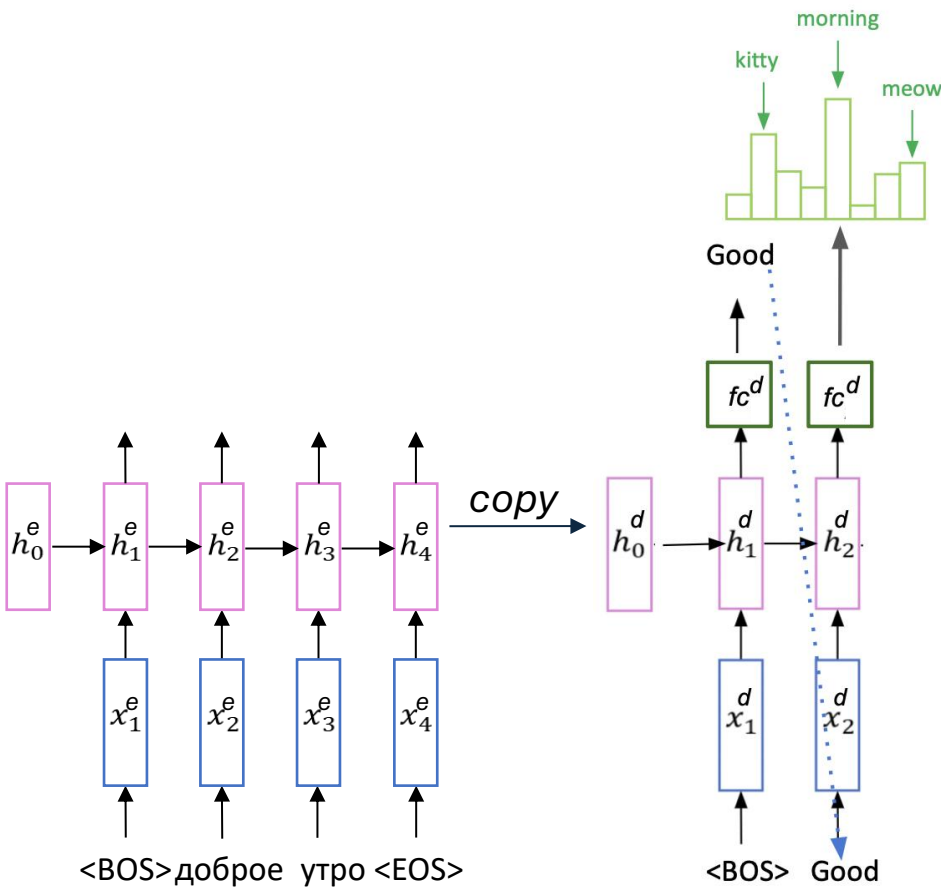




RNN

Source embeddings

Source input



распределение вероятностей

Softmax

FC слой

RNN декодера

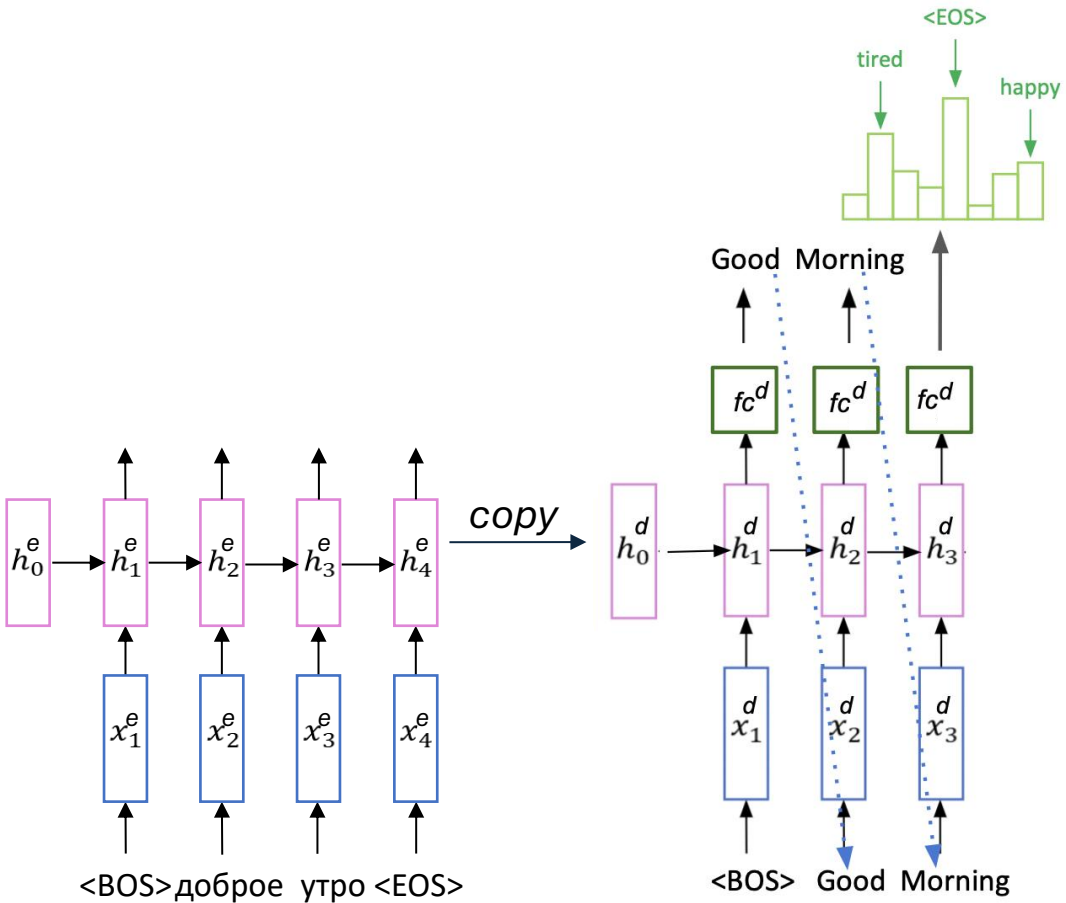
Target embeddings

сгенерированные токены

RNN

Source embeddings

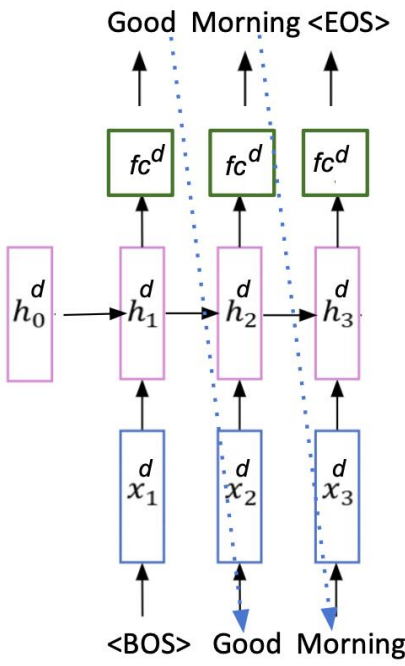
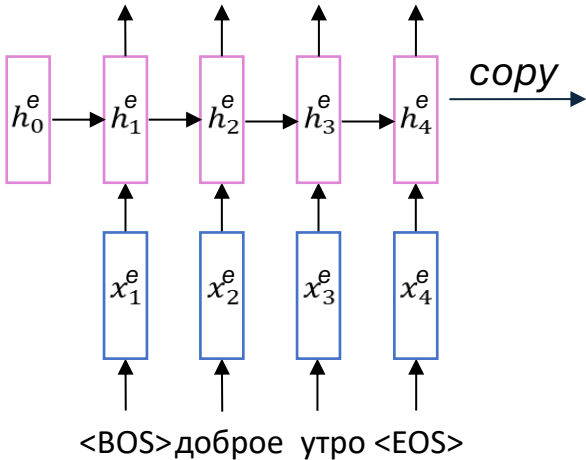
Source input



RNN

Source embeddings

Source input



сгенерированные токены

Softmax

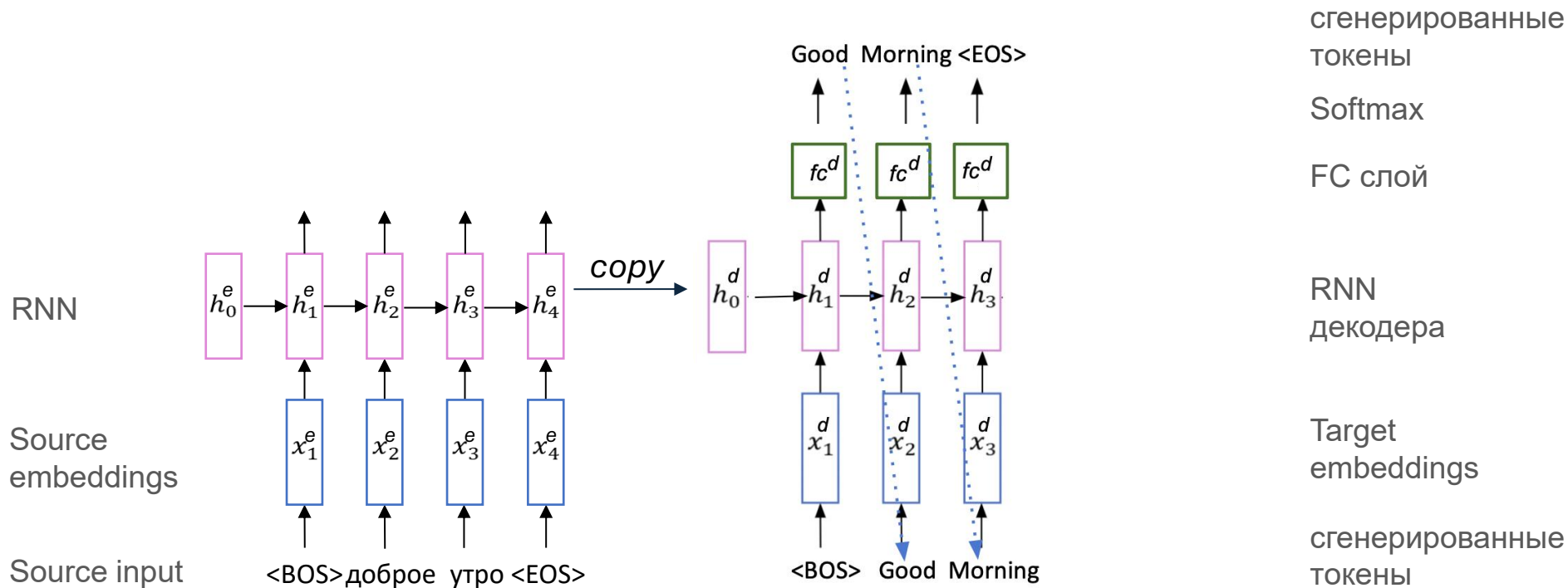
FC слой

RNN декодера

Target embeddings

сгенерированные токены

Такая архитектура модели
называется **Seq2Seq**
(sequence-to-sequence)



Обучение модели перевода

Чтобы обучить нашу модель, нам нужны данные.

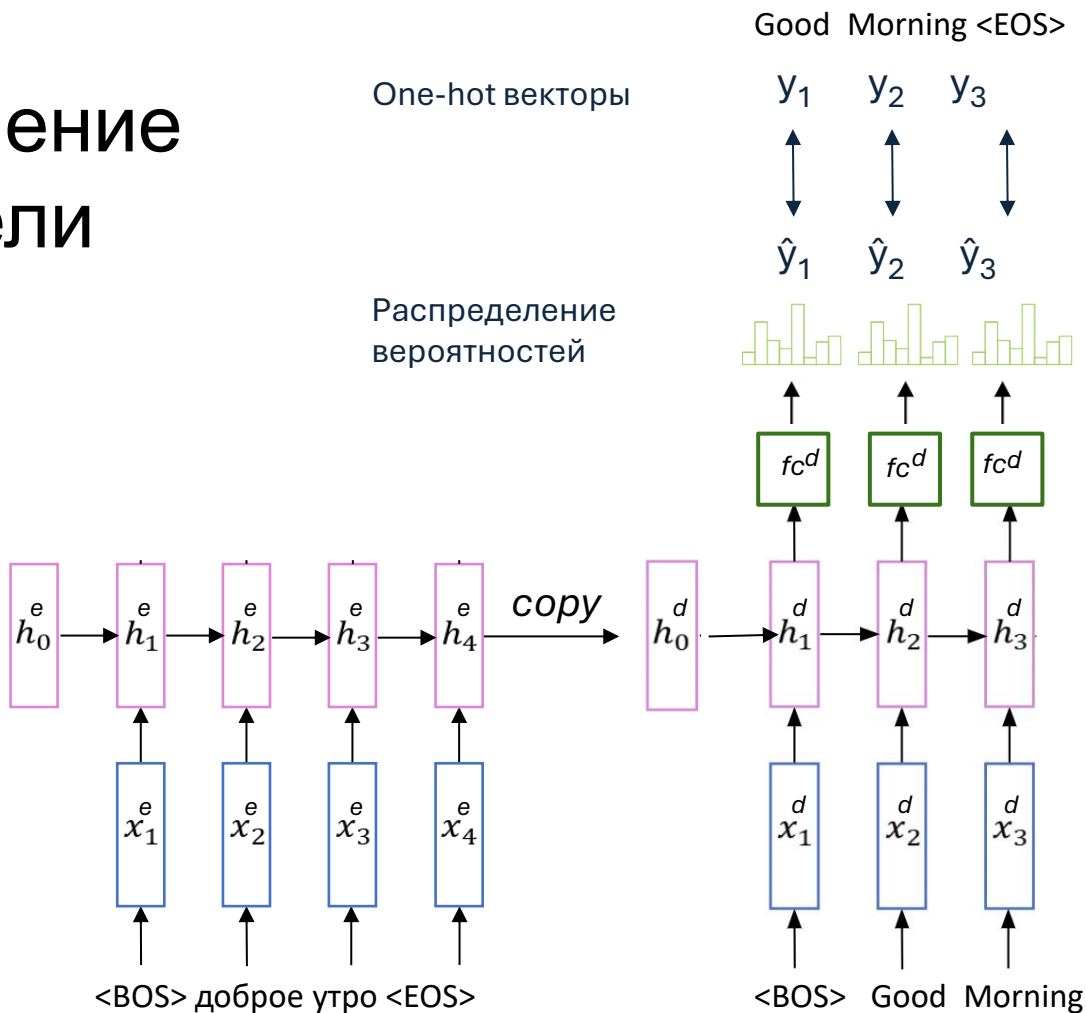
Данные для задачи машинного перевода —

это корпус параллельных текстов, т.е. набор пар вида:

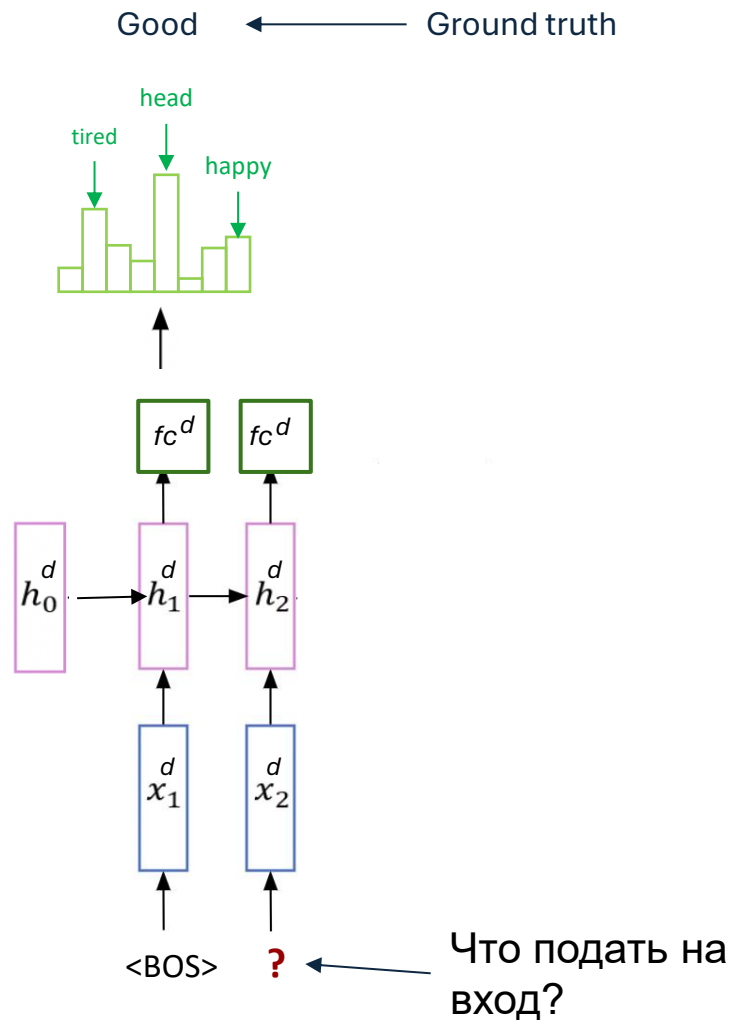
(source_sentence, target_sentence)

(доброе утро, good morning)

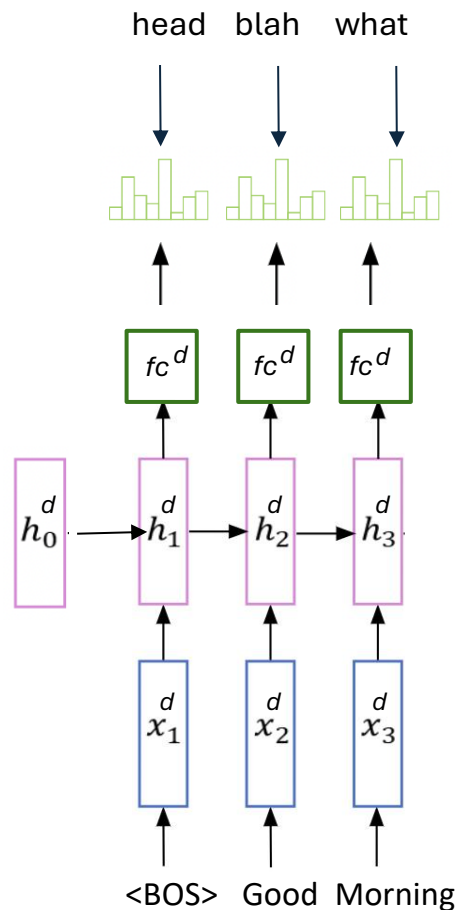
Обучение модели



Обучение модели



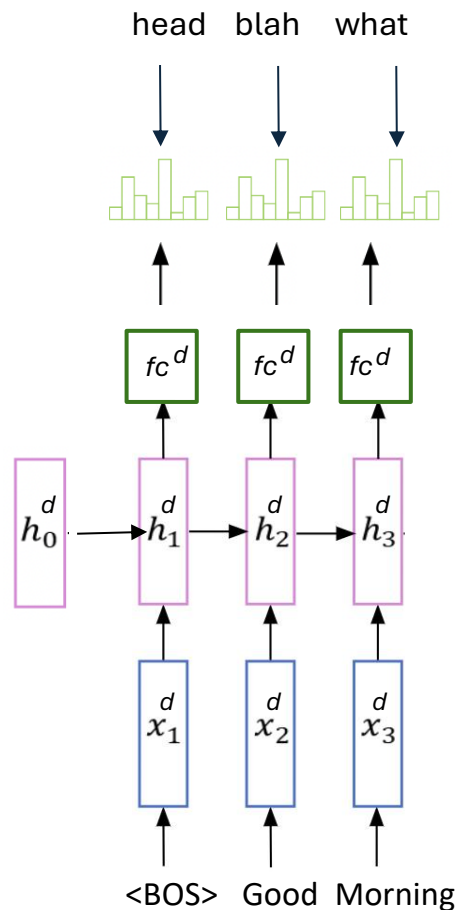
Обучение модели



Teacher forcing:

Подаем на вход декодеру токены, которые модель должна сгенерировать в идеале

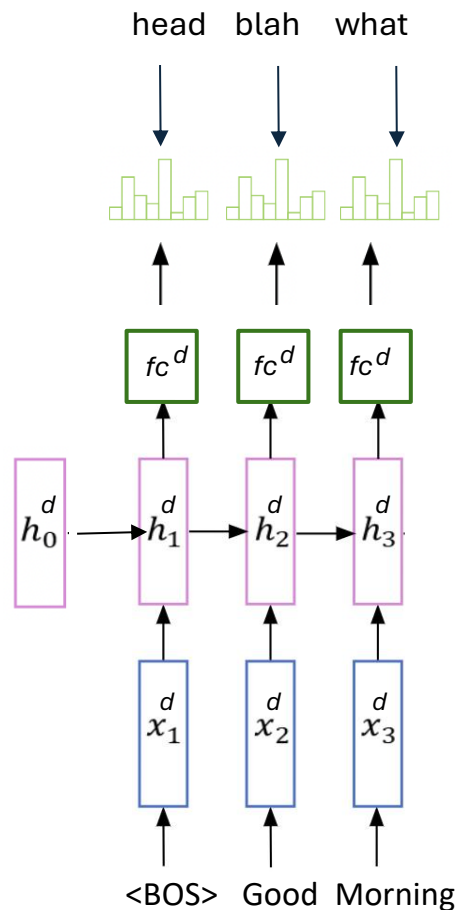
Обучение модели



Teacher forcing:

Можно использовать teacher forcing с вероятностью p на каждой итерации обучения, начиная с $p=1$ в начале обучения и постепенно уменьшая p

Обучение модели



Teacher forcing:

Во время инференса
использовать teacher
forcing не получится

Обучение модели

Нюансы обучения:

- У Encoder и Decoder разные словари (т.к языки разные);
- Для обоих словарей можно использовать subword tokenization;
- При генерации можно использовать beam search

Метрики качества для МТ

Как оценивать качество перевода?

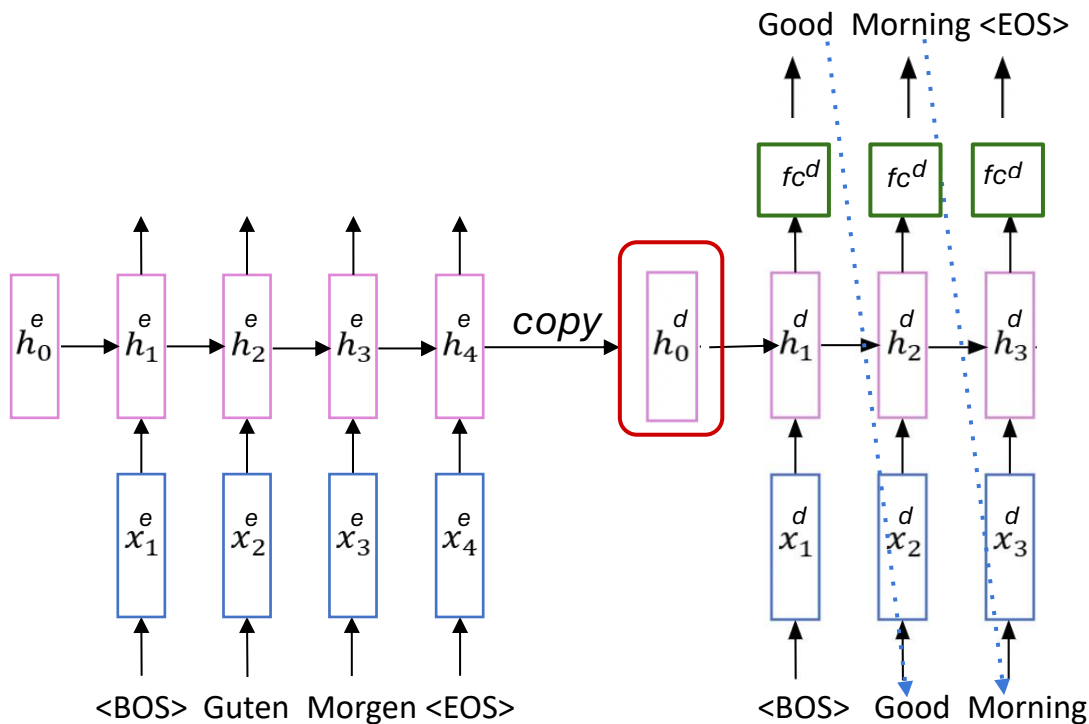
При обучении мы используем кросс-энтропию, но она слабо коррелирует с реальным качеством перевода.

Самая распространенная метрика оценки качества перевода — BLEU

Обучение модели

У построенной нами архитектуры есть недостатки

RNN for MT



Недостатки:

- Encoder RNN может забывать информацию из начала предложения;
- Вся информация о входном предложении содержится в одном векторе h_d^0

RNN for MT

Как можно бороться с этими проблемами:

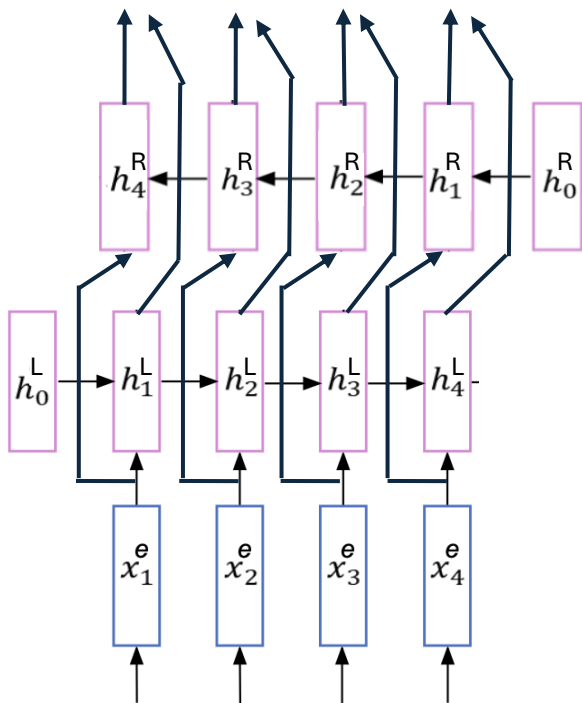
- Использовать GRU/LSTM;
- Использовать bidirectional RNN

Но ничего из этого не поможет решить проблему достаточно хорошо

Bidirectional RNN

Один bidirectional слой имеет два вектора скрытого состояния.

Один “читает” последовательность слева направо, второй — справа налево



Итоги раздела

В этом разделе мы разобрали:

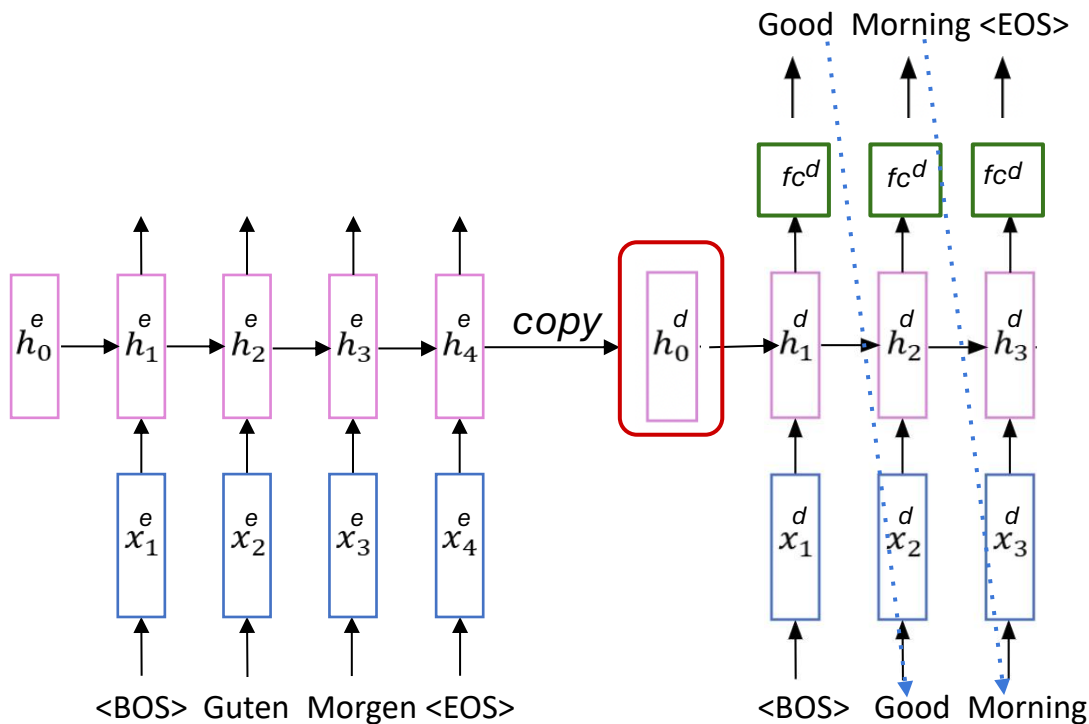
- Задачу машинного перевода;
- RNN-архитектуру Seq2Seq для решения задачи перевода;
- Принцип и нюансы обучения модели Seq2Seq для перевода;
- Недостатки модели Seq2Seq, основанной на RNN.

Механизм Attention

План раздела

- Задача машинного перевода Механизм attention: идея
- Применение Attention для RNN-модели Seq2Seq для задачи машинного перевода;
- Разные варианты Attention

RNN для МТ: напоминание



Недостатки:

- Encoder RNN может забывать информацию из начала предложения;
- Вся информация о входном предложении содержится в одном векторе h_d^0

Attention: идея

Допустим, нам надо перевести следующее предложение на английский язык:

*Высокий статный мужчина в шляпе с прекрасным ярким попугаем на плече
только что зашел в здание на углу Старого Косого переулка*

Attention: идея

Допустим, нам надо перевести следующее предложение на английский язык:

*Высокий статный мужчина в шляпе с прекрасным ярким попугаем на плече
только что зашел в здание на углу Старого Косого переулка*

Как работает наша RNN-модель:

- Читает это предложение один раз и “запоминает” его в вектор скрытого состояния;
- Генерирует предложение-перевод

Attention: идея

Допустим, нам надо перевести следующее предложение на английский язык:

*Высокий статный мужчина в шляпе с прекрасным ярким попугаем на плече
только что зашел в здание на углу Старого Косого переулка*

Что бы делали мы (люди):

- Во время генерации перевода мы бы несколько раз смотрели на разные части исходного предложения, чтобы найти те части, на которые стоит обратить внимание в данный момент времени, и лучше их запомнить

Attention: идея

Допустим, нам надо перевести следующее предложение на английский язык:

*Высокий статный мужчина в шляпе с прекрасным ярким попугаем на плече
только что зашел в здание на углу Старого Косого переулка*

Механизм Attention — это то, что поможет декодеру модели “подглядывать” в нужные части исходного предложения во время генерации.

Attention

Идея Attention была впервые предложена
в 2014-2015 году в [этой](#) статье

Published as a conference paper at ICLR 2015

NEURAL MACHINE TRANSLATION BY JOINTLY LEARNING TO ALIGN AND TRANSLATE

Dzmitry Bahdanau

Jacobs University Bremen, Germany

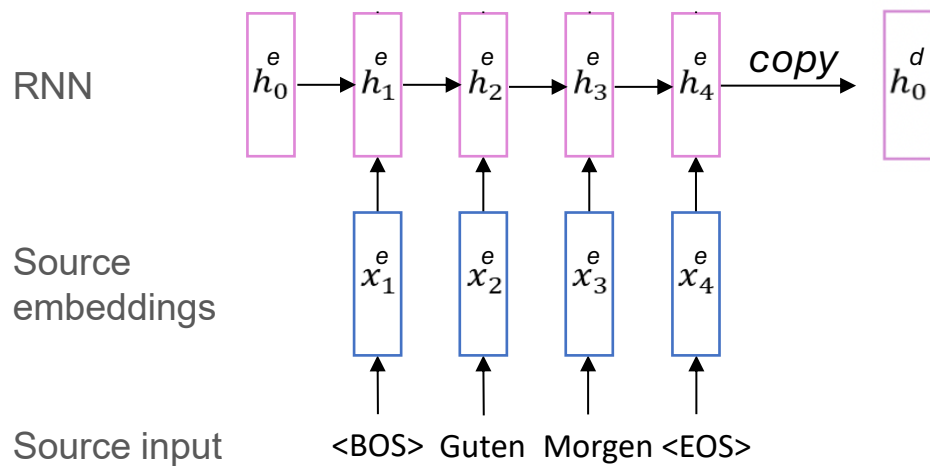
KyungHyun Cho **Yoshua Bengio***

Université de Montréal

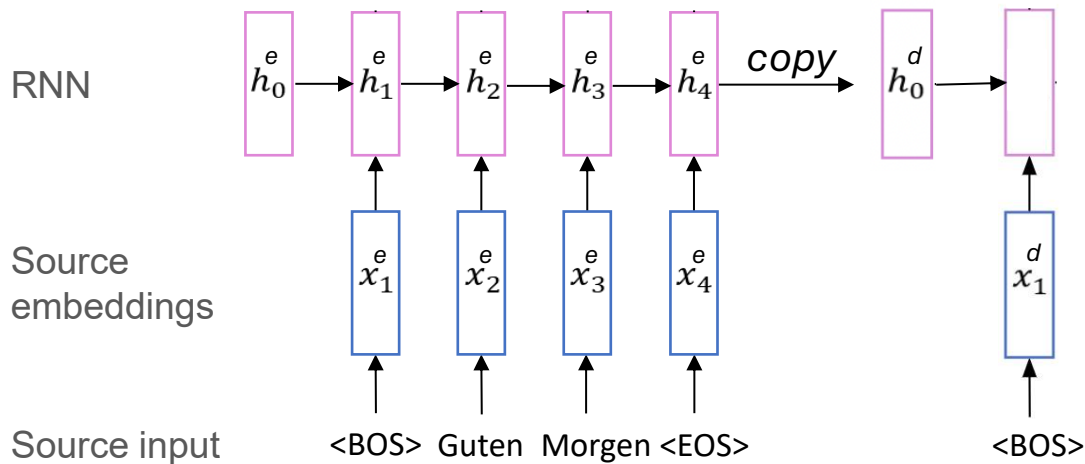
ABSTRACT

Neural machine translation is a recently proposed approach to machine translation. Unlike the traditional statistical machine translation, the neural machine translation aims at building a single neural network that can be jointly tuned to maximize the translation performance. The models proposed recently for neural machine translation often belong to a family of encoder-decoders and encode a source sentence into a fixed-length vector from which a decoder generates a translation. In this paper, we conjecture that the use of a fixed-length vector is a bottleneck in improving the performance of this basic encoder-decoder architecture, and propose to extend this by allowing a model to automatically (soft-)search for parts of a source sentence that are relevant to predicting a target word, without having to form these parts as a hard segment explicitly. With this new approach, we achieve a translation performance comparable to the existing state-of-the-art phrase-based system on the task of English-to-French translation. Furthermore, qualitative analysis reveals that the (soft-)alignments found by the model agree well with our intuition.

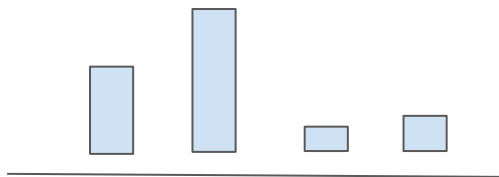
Энкодер будет работать точно так же, как и раньше



Перед тем, как обновить скрытое состояние декодера, нам нужно “обратить внимание” на входную последовательность



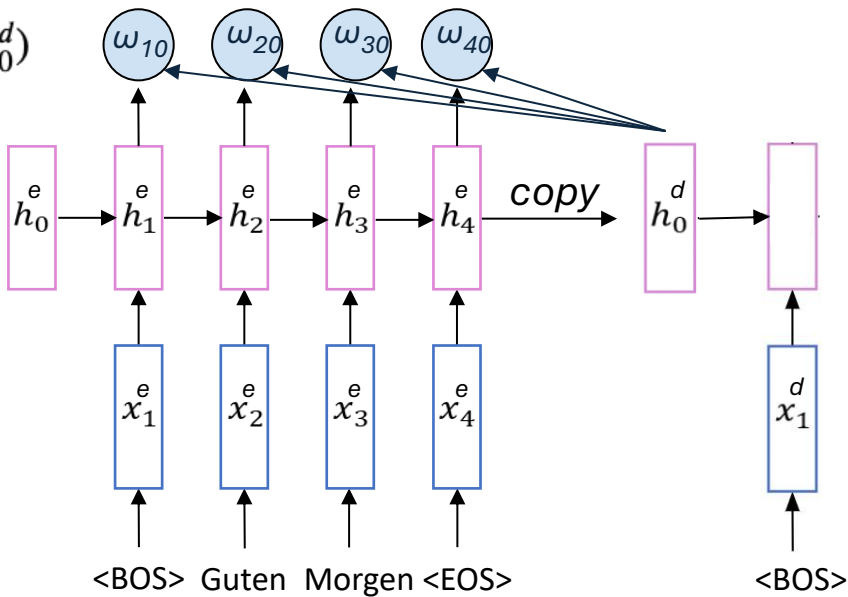
0.56 1.42 -0.45 -0.08



ω_{i0} — это вес (важность) i -го входного токена для текущего состояния декодера

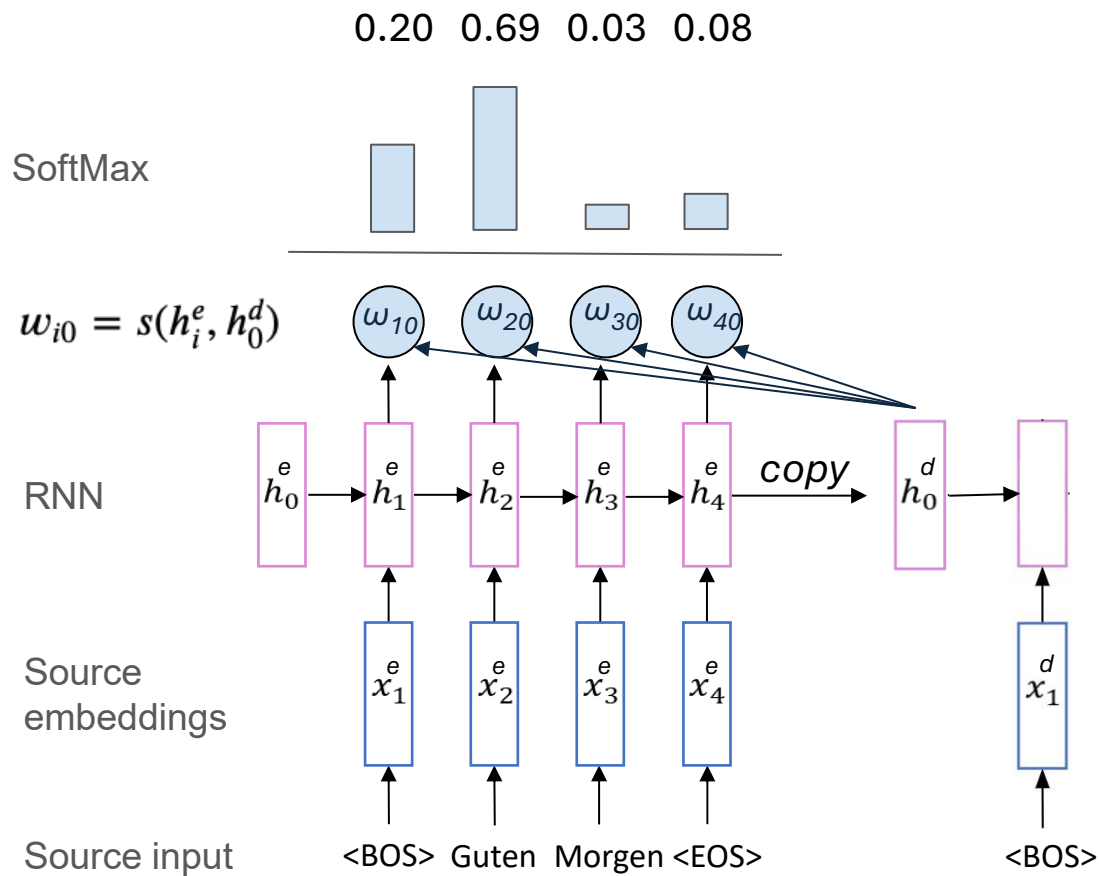
$$w_{i0} = s(h_i^e, h_0^d)$$

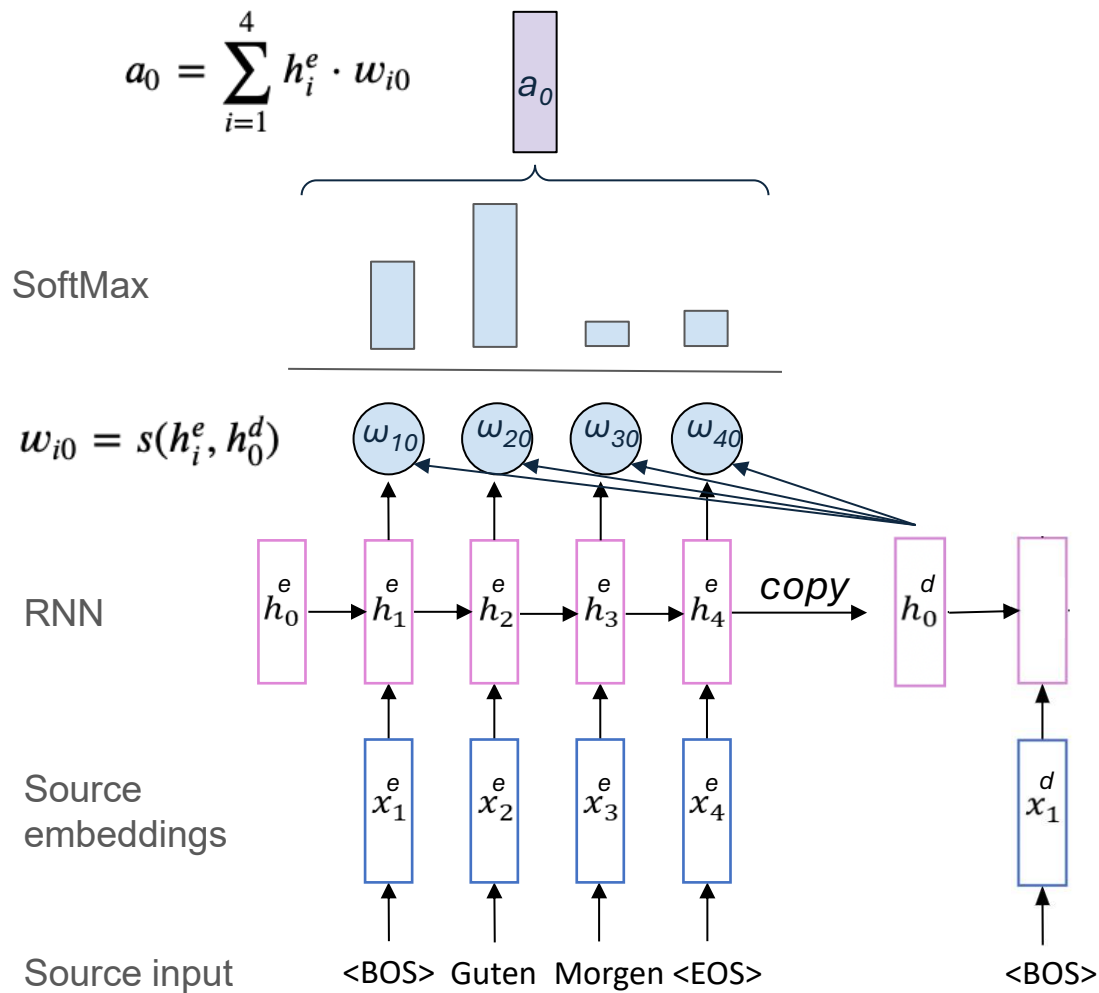
RNN

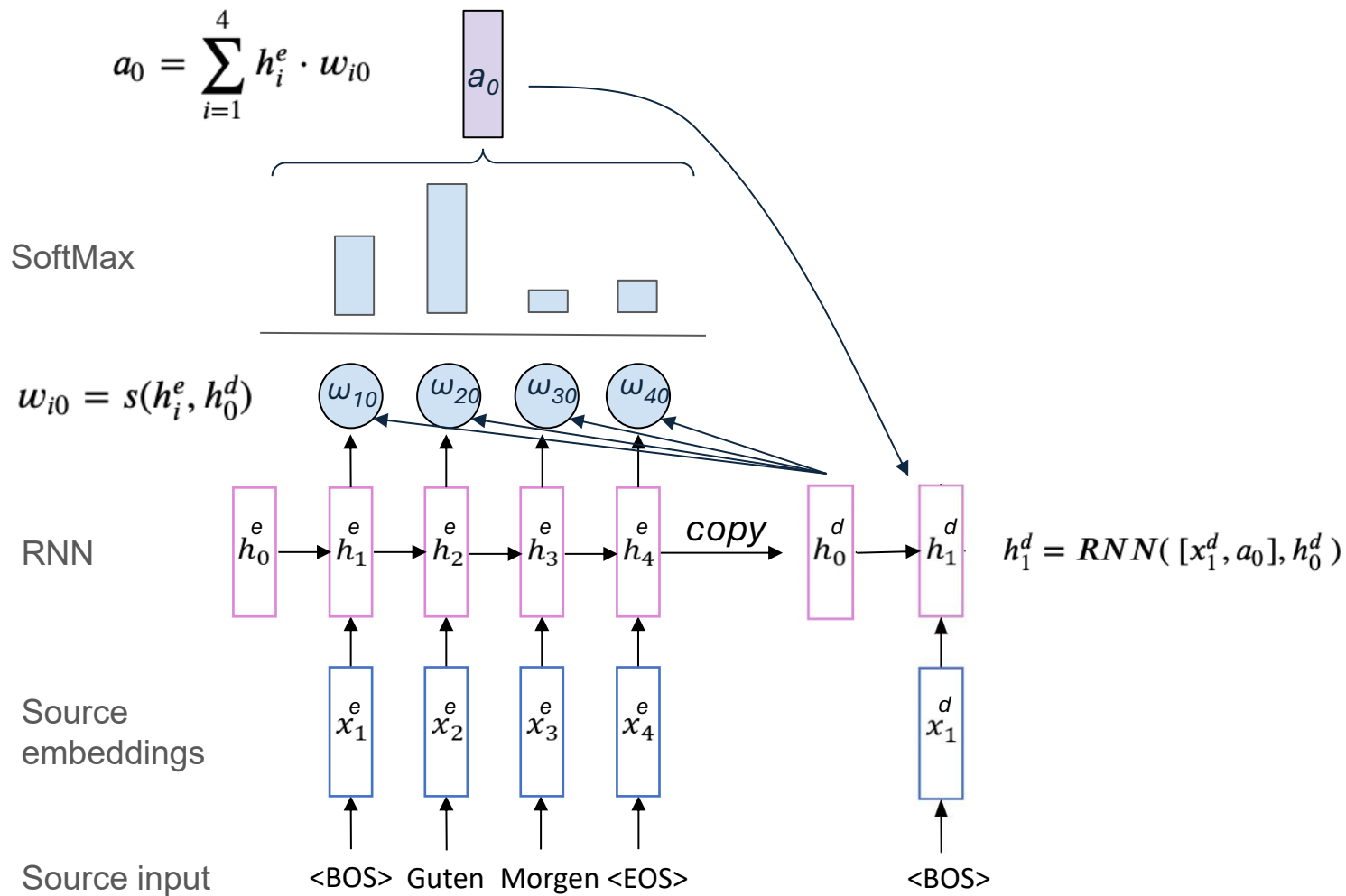


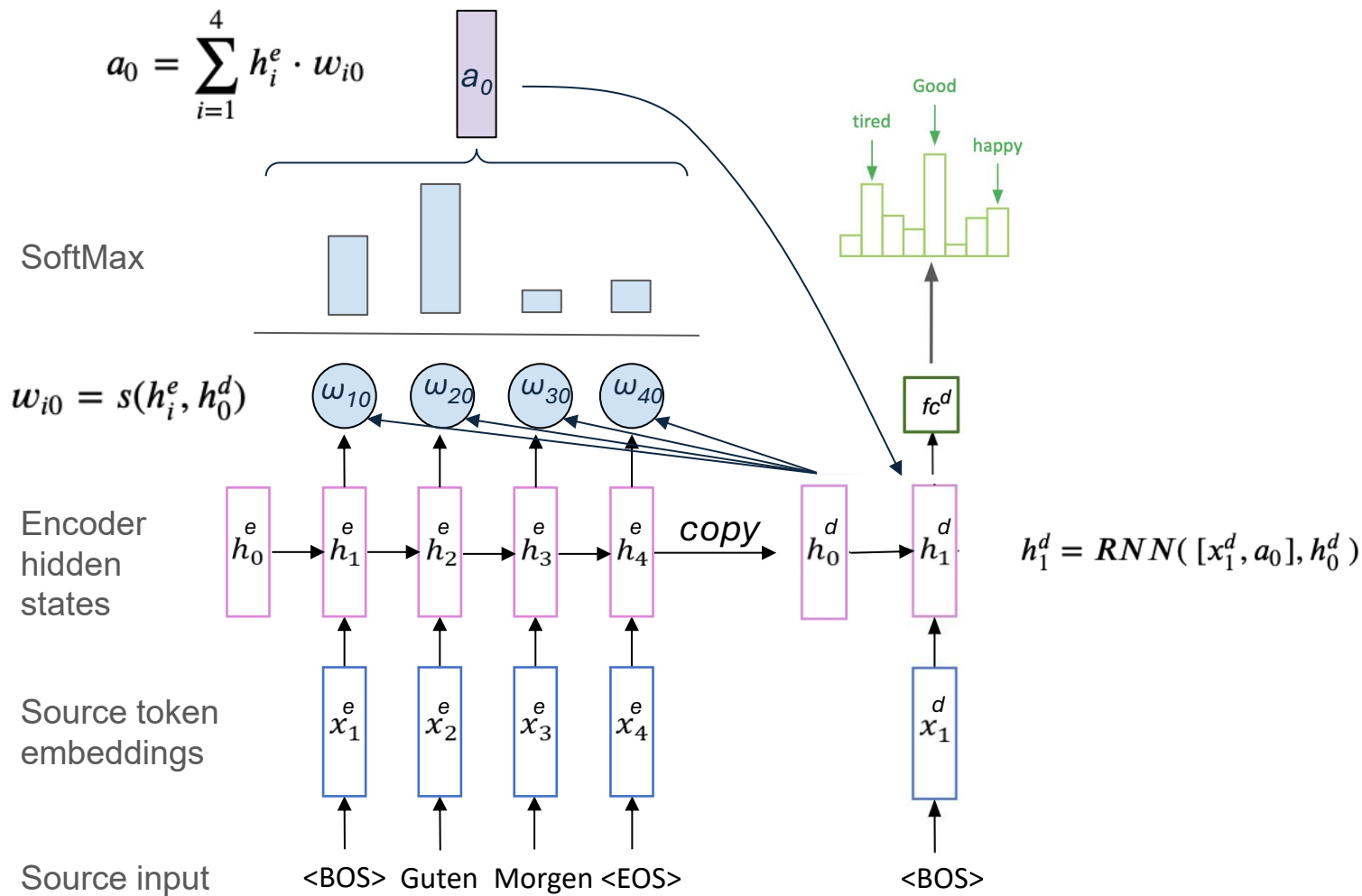
Source embeddings

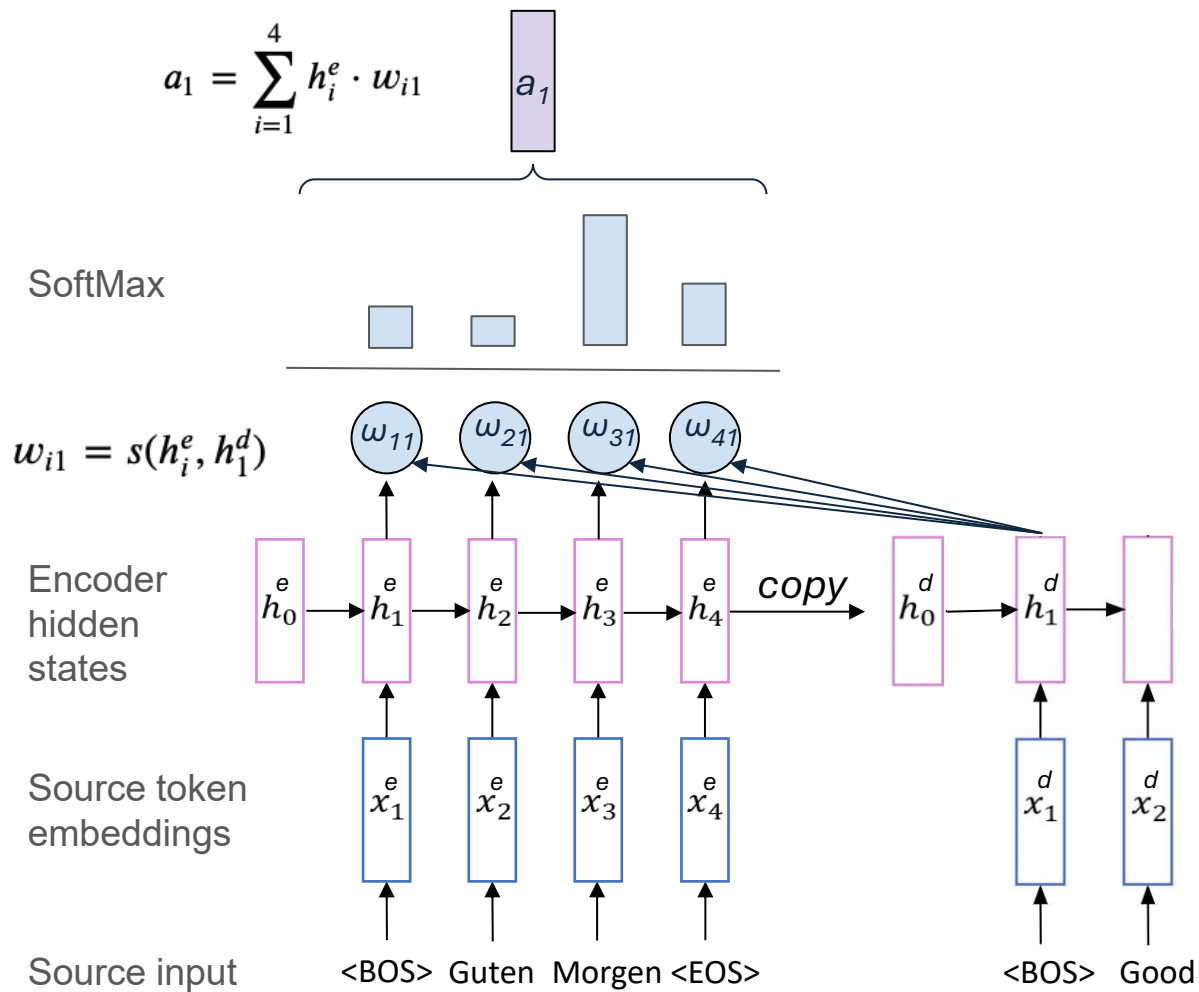
Source input

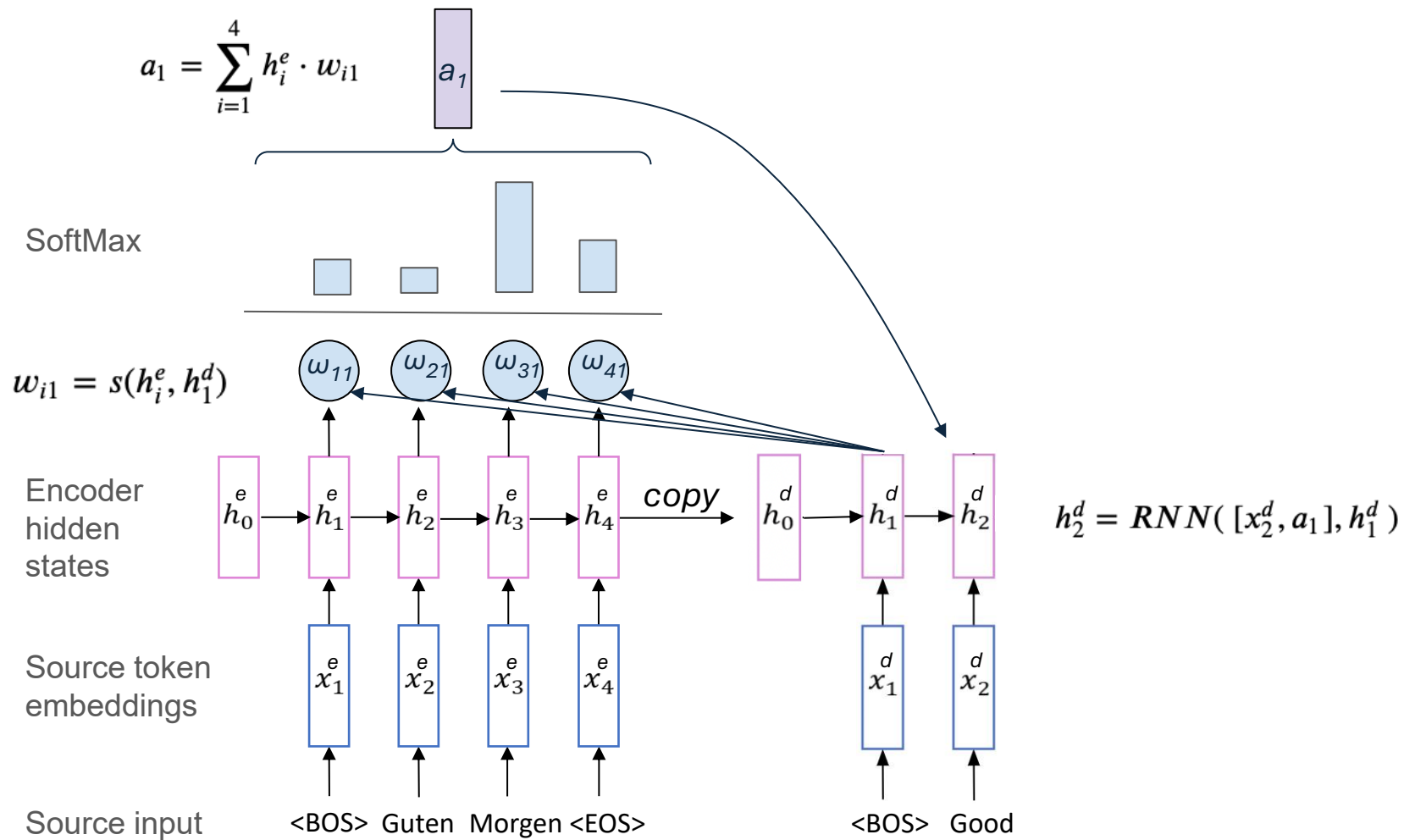


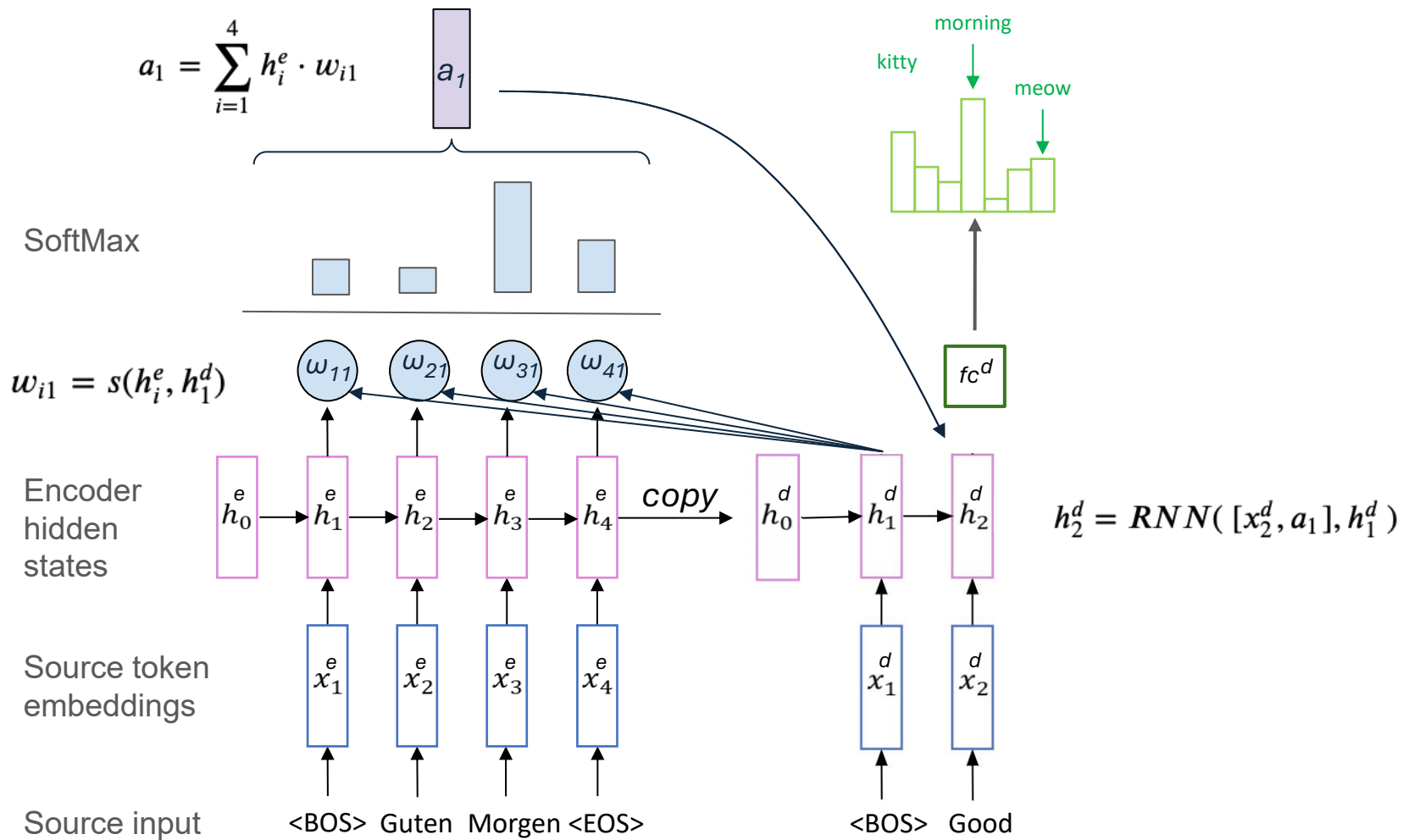


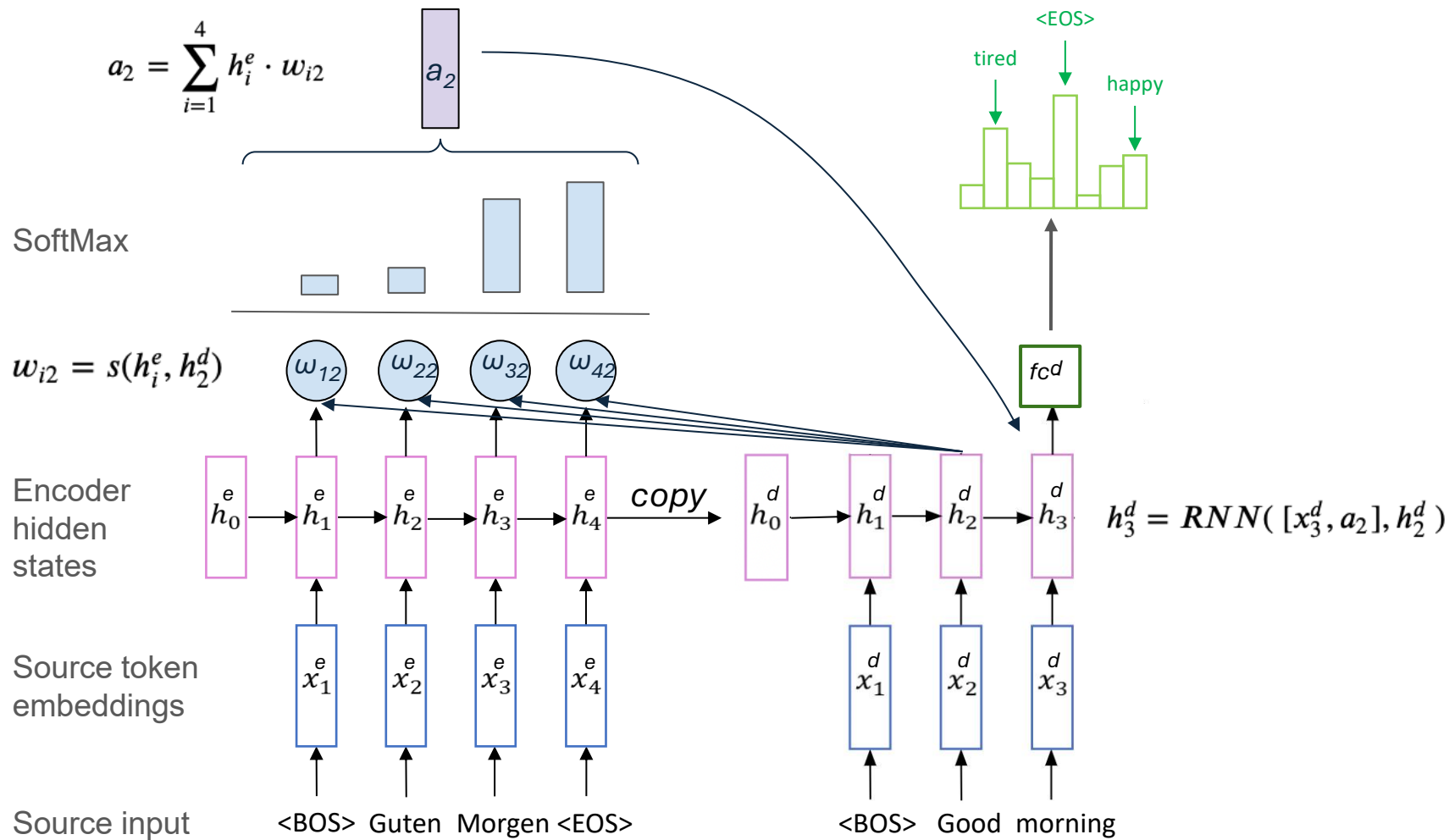


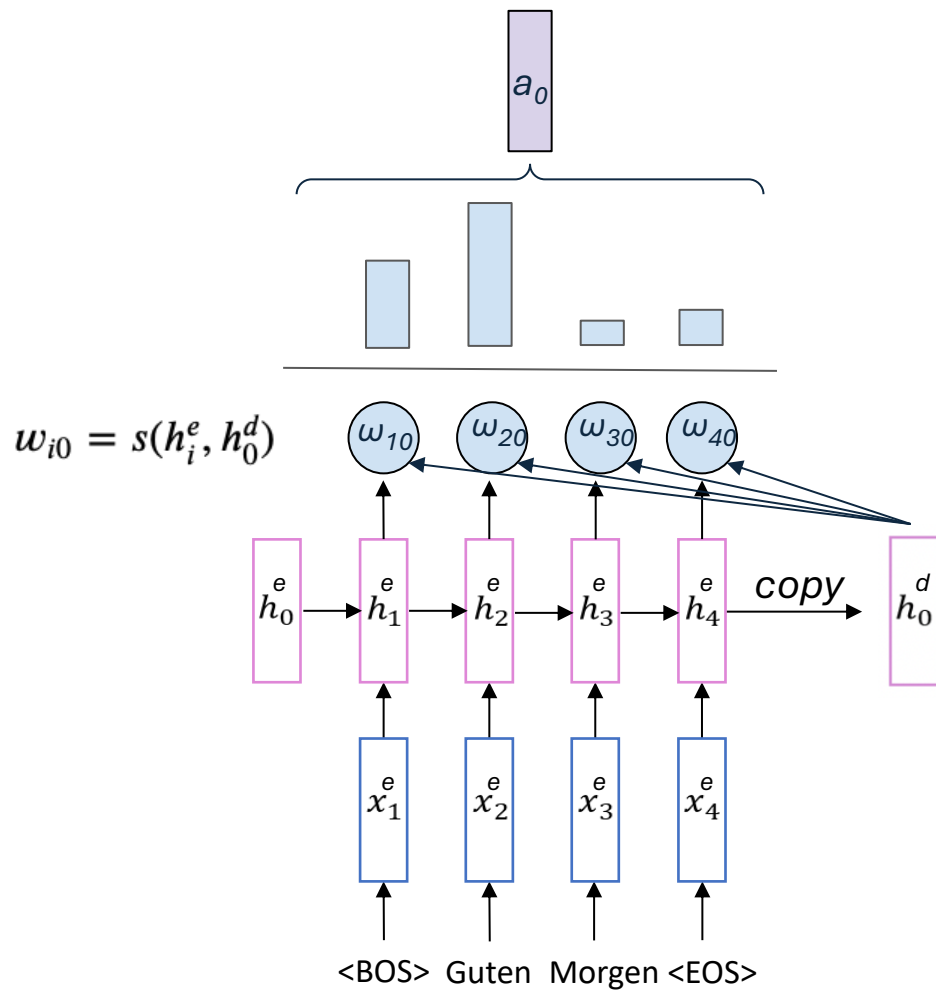












Функция s может быть обучаемой или не обучаемой.

Если она обучаемая, то ее веса обучаются совместно со всеми параметрами сети.

Варианты функций Attention

$$w = s(h_1, h_2)$$

- Dot product attention:
(без обучаемых параметров)

$$s(h_1, h_2) = h_1^T h_2$$

- Однослойная нейросеть
(с обучаемыми параметрами)

$$s(h_1, h_2) = \sigma(W[h_1, h_2])$$

- Многие другие

Эффект применения Attention

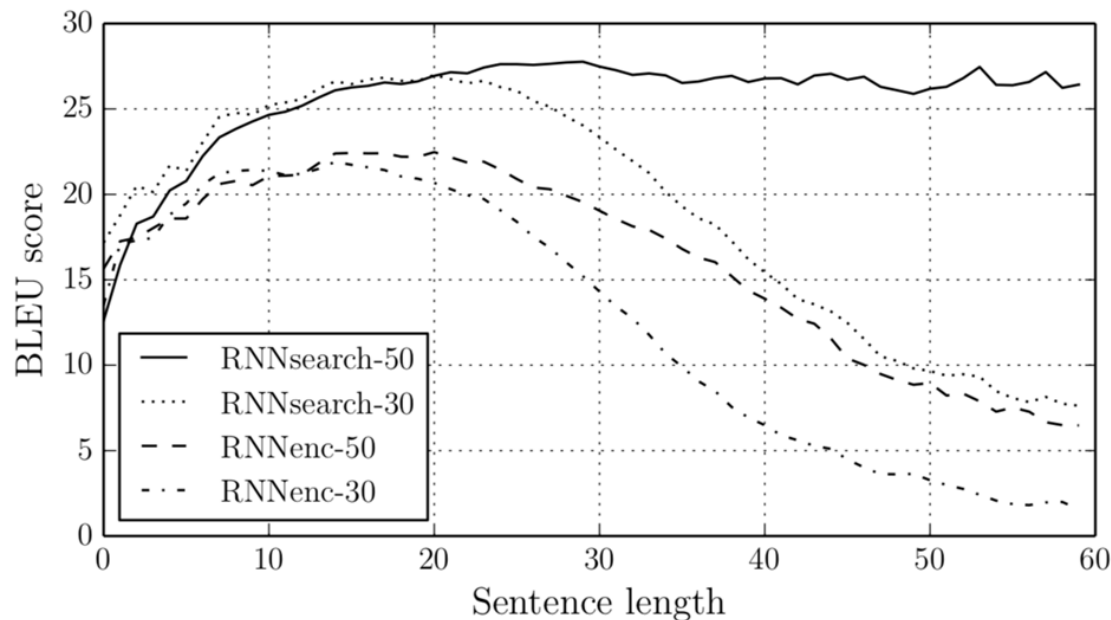
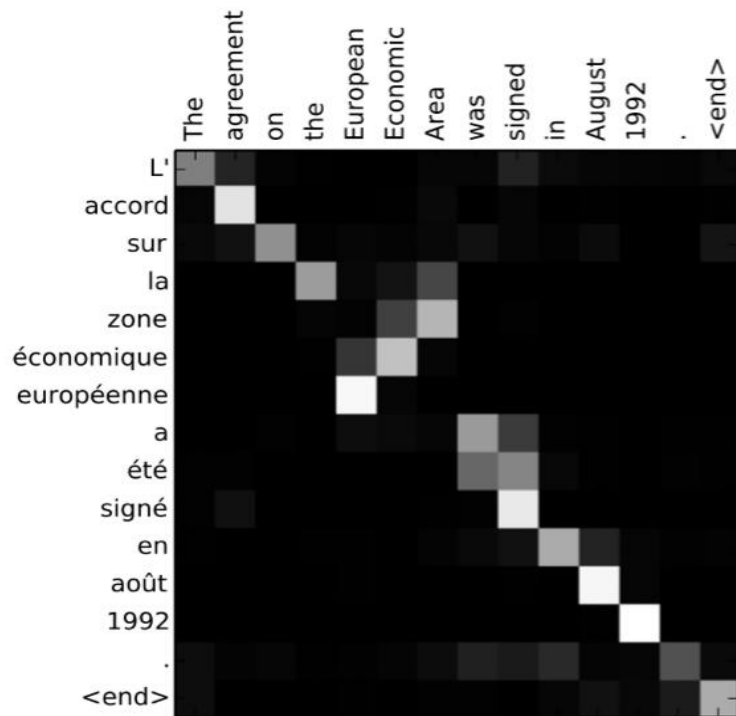
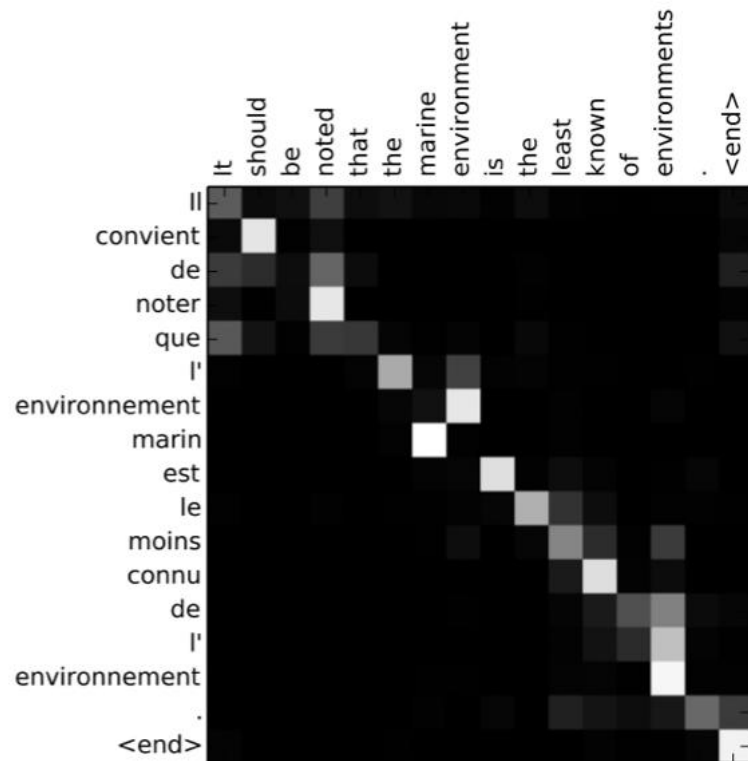


Figure 2: The BLEU scores of the generated translations on the test set with respect to the lengths of the sentences. The results are on the full test set which includes sentences having unknown words to the models.

Attention visualization



Attention visualization



Революция после Attention

Изобретение Attention было поворотным моментом в задаче машинного перевода и всей области NLP.

В 2017 году Google представили новую архитектуру нейросети, которую называли Transformer. Основой ее был механизм Attention. Transformer сотворили революцию в NLP и потом были успешно адаптированы и к другим доменам.

Итоги раздела

В этом разделе:

- Идея механизма Attention;
- Attention для RNN-модели Seq2Seq;
- Разные варианты Attention;
- Attention может быть интерпретируемым.

Transformer architecture

Encoder block

В этом разделе

- **Идея архитектуры Transformer**
- **Устройство Transformer Encoder**
- Устройство Transformer Decoder
- QKV Attention

Недостатки RNN

Encoder на основе RNN очень медленный:

- Forward pass работает медленно, так как обрабатывает входящие токены последовательно;
- Backward pass работает медленно, так как происходит распространение градиентов сквозь время

У RNN возникают проблемы во время обучения:

- Затухание/взрыв градиентов;
- Плохая утилизация GPU

Недостатки RNN

Нужны ли нам все еще RNN?

Мы использовать RNN для решения задачи машинного перевода, потому что их структура позволяет собирать информацию из входной последовательности и передавать ее декодеру.

Но сейчас у нас есть механизм Attention, который позволяет декодеру в любой момент времени “смотреть” на любую часть исходного предложения.

Идея Transformer

Transformer — архитектура, основа которой — механизм Attention.

У Transformer нет RNN-слоев.

Идея Transformer

Transformer впервые предложили в 2017 году в статье от Google “**Attention is all you need**”

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

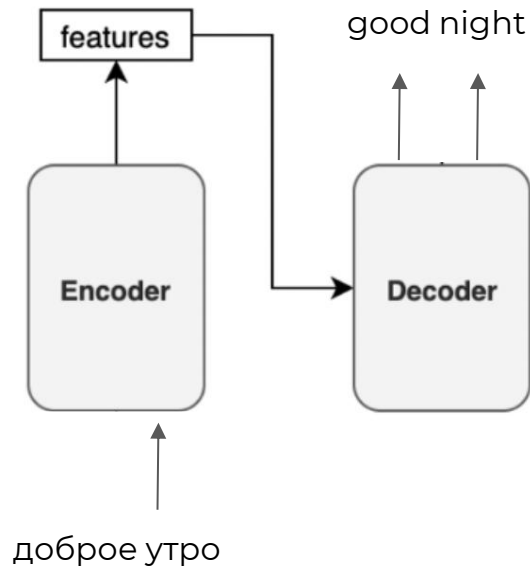
Łukasz Kaiser*
Google Brain
lukaszkaizer@google.com

Illia Polosukhin* †
illia.polosukhin@gmail.com

Abstract

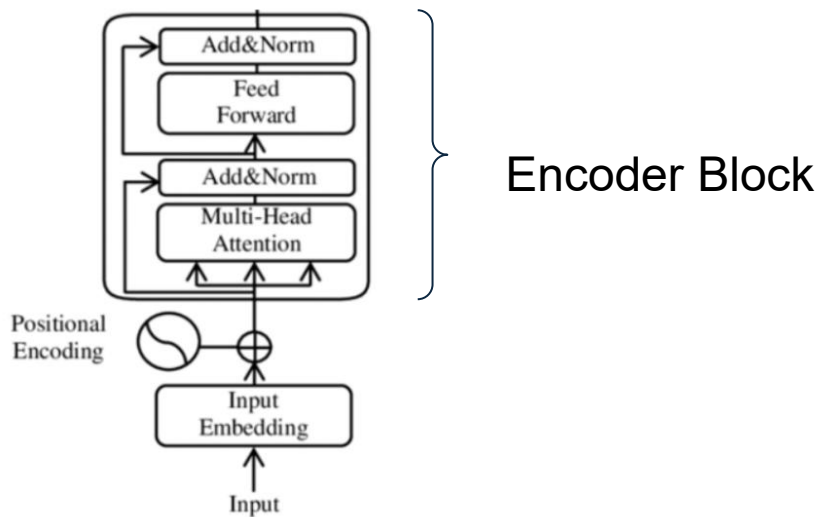
Идея Transformer

У Transformer архитектура тоже имеет тип Encoder-Decoder

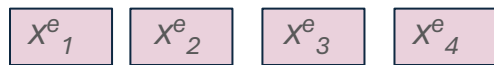


Transformer Encoder

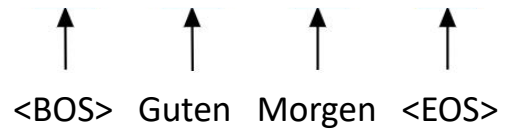
Encoder обрабатывает токены входящей последовательности **параллельно**



Source
embeddings

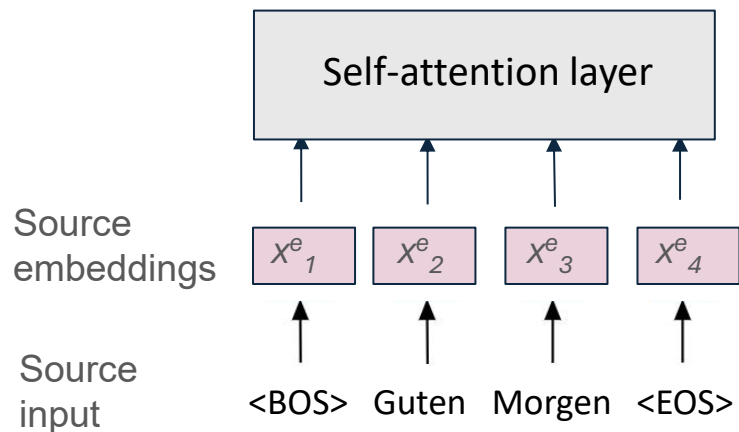


Source
input



Слой self-attention вычисляет дополнительную информацию для эмбединга каждого токена на основе его контекста (т.е. эмбедингов всех других токенов)

“Sport is good, it helps a lot”



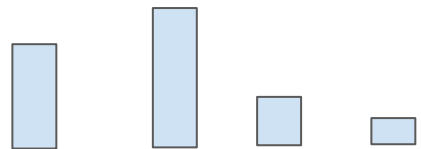
$$a_1^e = \sum_{i=1}^4 w_{1i} x_i^e$$

a_1^e

Агрегация

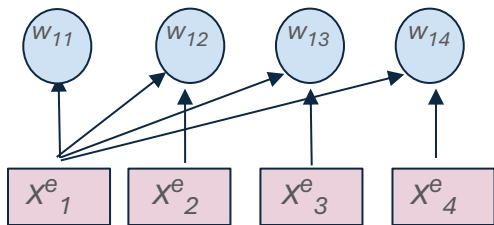
0.4 0.55 0.1 0.05

SoftMax



Self-attention

$$w_{1i} = s(x_1^e, x_i^e)$$



Source embeddings

Source input

$\uparrow$ $\uparrow$ $\uparrow$ $\uparrow$
 <BOS> Guten Morgen <EOS>

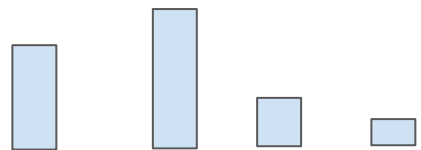
$$a_2^e = \sum_{i=1}^4 w_{2i} x_i^e$$

$$a_1^e \quad a_2^e$$

Агрегация

0.25 0.45 0.25 0.05

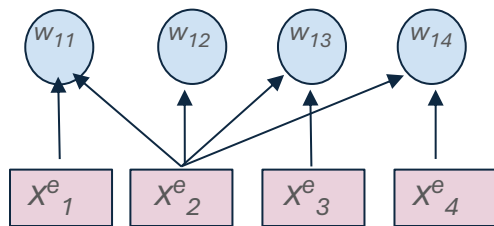
SoftMax



Self-attention

$$w_{2i} = s(x_2^e, x_i^e)$$

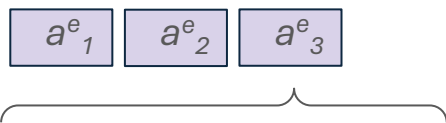
Source embeddings



Source input

<BOS> Guten Morgen <EOS>

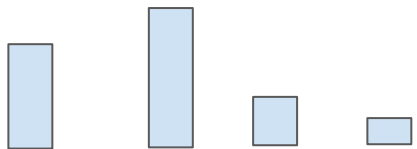
$$a_3^e = \sum_{i=1}^4 w_{3i} x_i^e$$



Агрегация

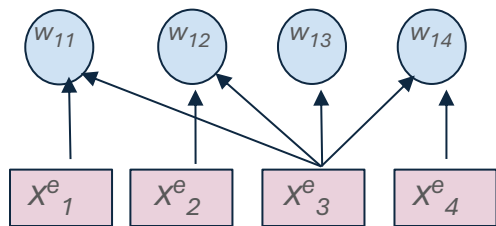
0.4 0.5 0.05 0.05

SoftMax



Self-attention

$$w_{3i} = s(x_2^e, x_i^e)$$



Source embeddings

Source input

<BOS> Guten Morgen <EOS>

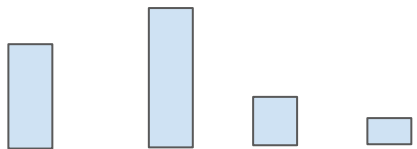
$$a_4^e = \sum_{i=1}^4 w_{4i} x_i^e$$



Агрегация

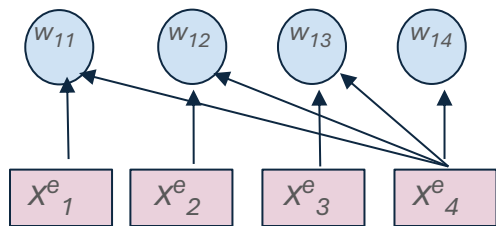
0.2 0.1 0.65 0.05

SoftMax



Self-attention

$$w_{4i} = s(x_4^e, x_i^e)$$

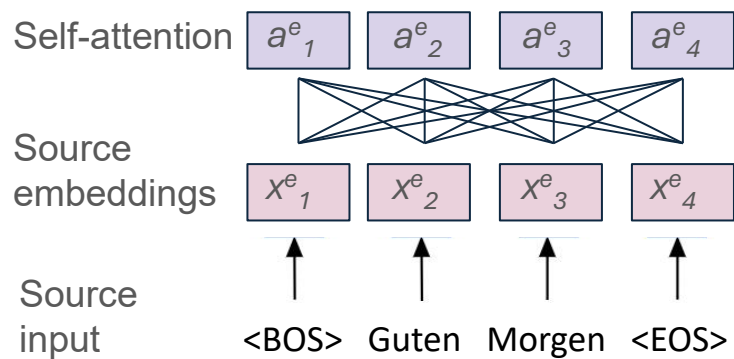


Source embeddings

Source input

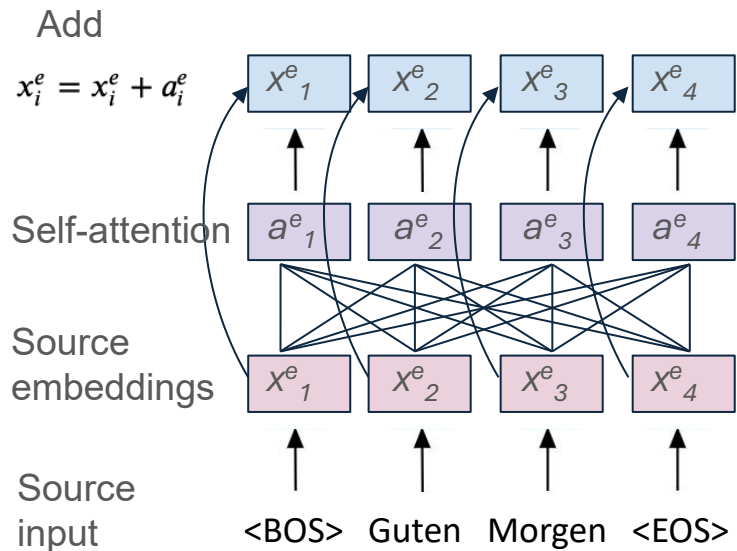
<BOS> Guten Morgen <EOS>

Self-Attention применяется ко всем
эмбедингам параллельно

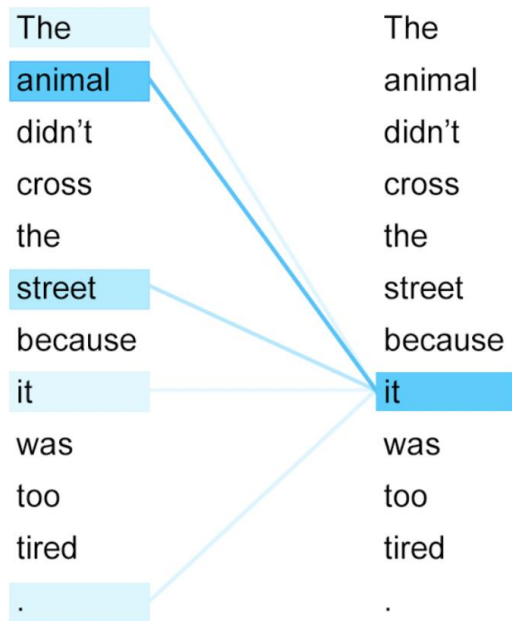


Слой self-attention вычисляет дополнительную информацию для эмбединга каждого токена на основе его контекста

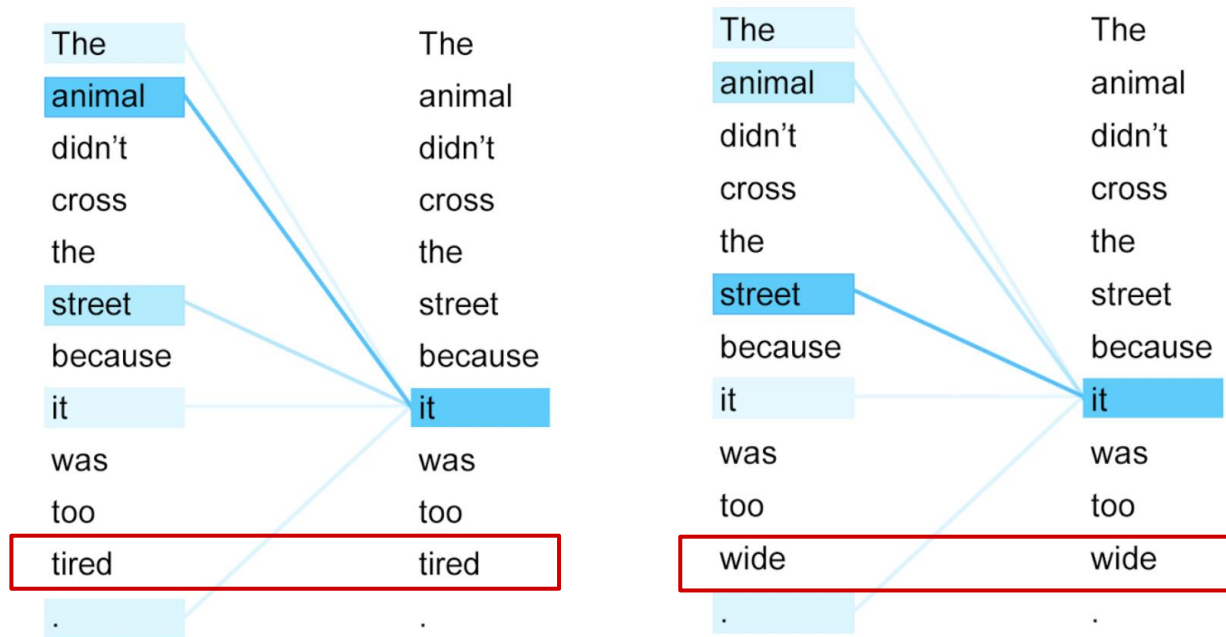
“Sport is good, it helps a lot”



Визуализация весов слоя self-attention

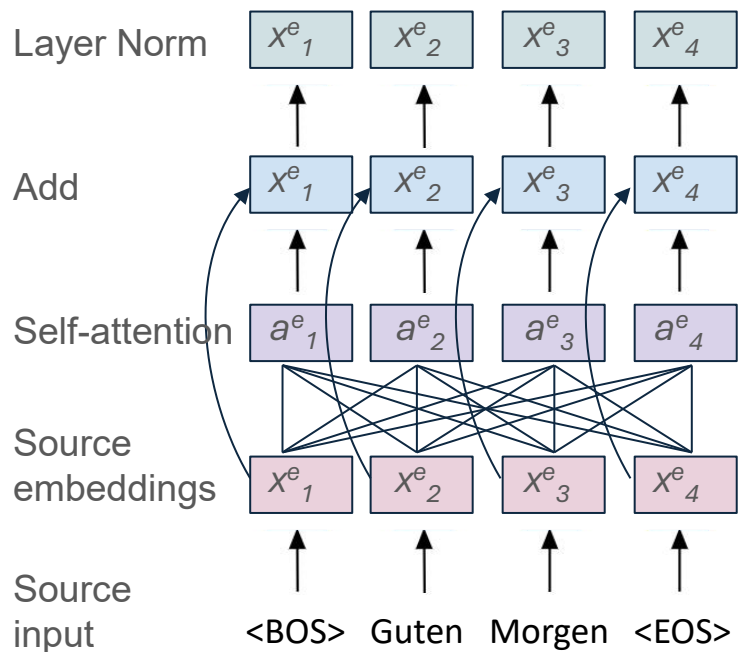


Визуализация весов слоя self-attention

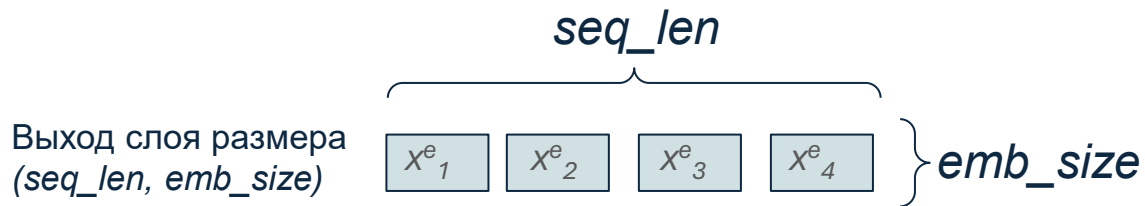


LayerNorm делает так, что выходы слоя всегда будут иметь одинаковые mean и variance.

Это делает обучение сети более стабильным



Layer Norm

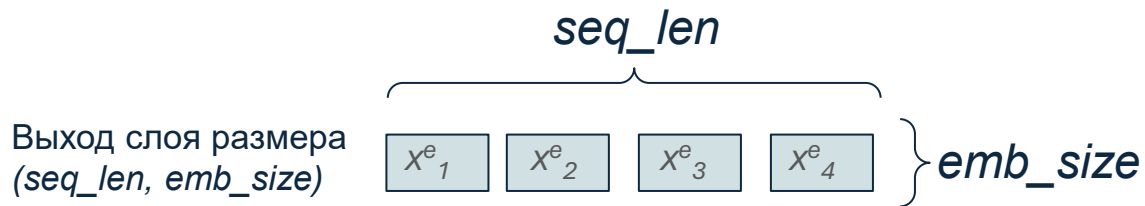


Считаем среднее и стандартное отклонение по всем координатам каждого эмбединга x_i^e :

$$\mu_i = \frac{1}{emb_size} \sum_{j=1}^{emb_size} x_j^e$$
$$\sigma_i = \sqrt{\frac{1}{emb_size} \sum_{j=1}^{emb_size} (x_j^e - \mu_i)^2}$$

$$1 \leq i \leq seq_len$$

Layer Norm



После этого обновляем элементы каждого эмбединга x^e_i следующим образом:

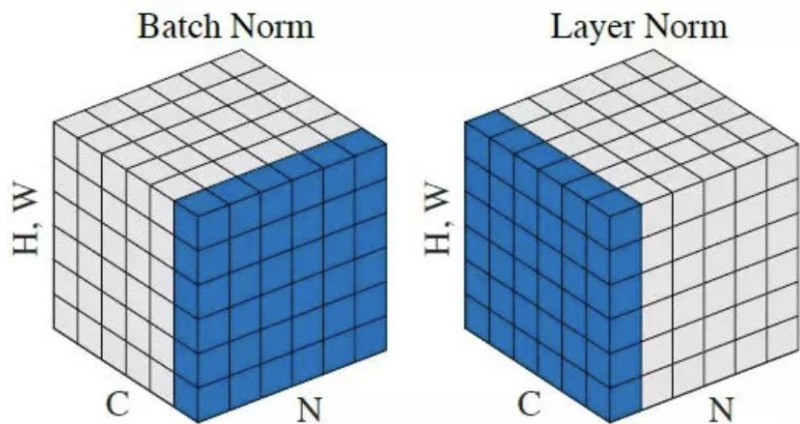
$$x^e_i = \frac{x^e_i - \mu_i}{\sigma_i + \epsilon} \cdot \gamma + \beta$$

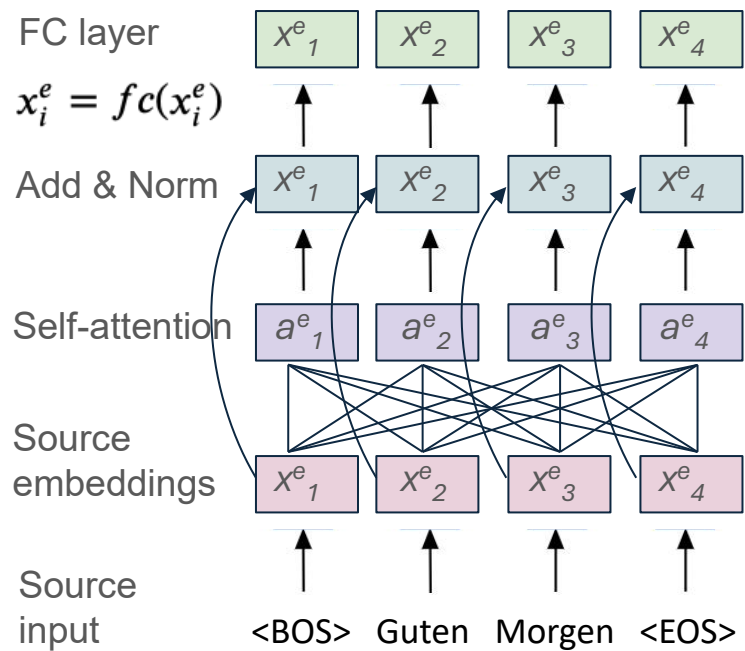
$$1 \leq i \leq seq_len$$

Здесь γ и β — обучаемые параметры

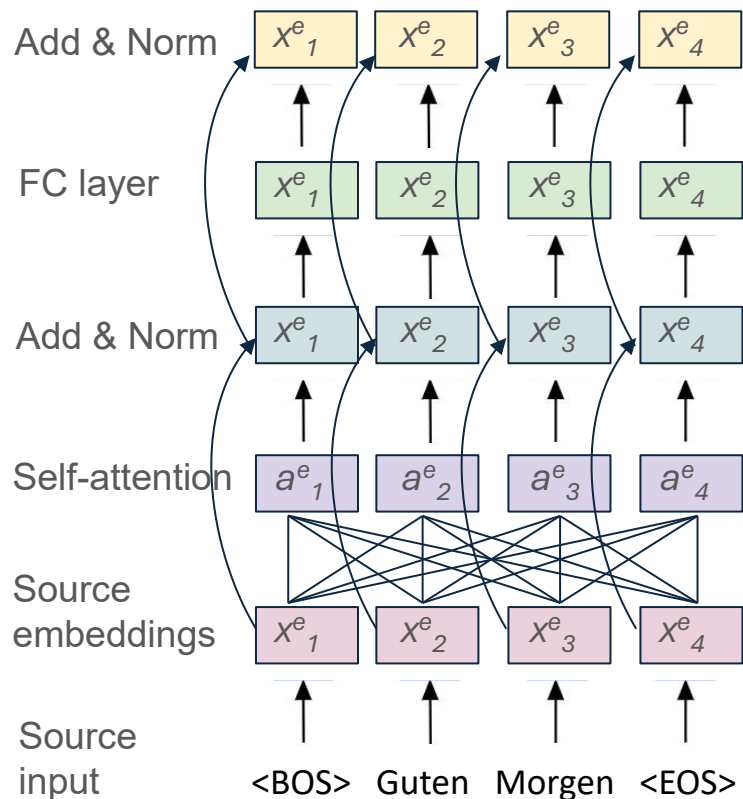
Layer Norm

LayerNorm можно лучше понять, если сравнить его с BatchNorm:





Transformer Encoder



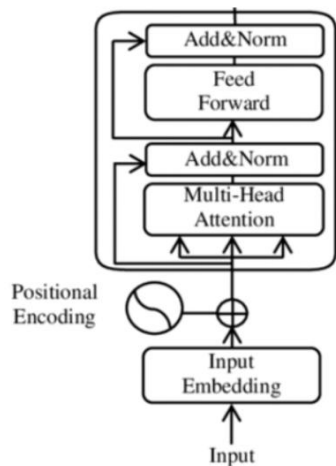
- Результат работы блока Encoder — обновленные эмбединги токенов.
- Каждый слой обрабатывает эмбединги параллельно
- Каждый слой, кроме слоя self-attention, обрабатывает эмбединги независимо
- Transformer Encoder может обрабатывать последовательности разной длины
- Слои в энкодере можно менять местами

Transformer Encoder

Мы почти собрали Encoder.

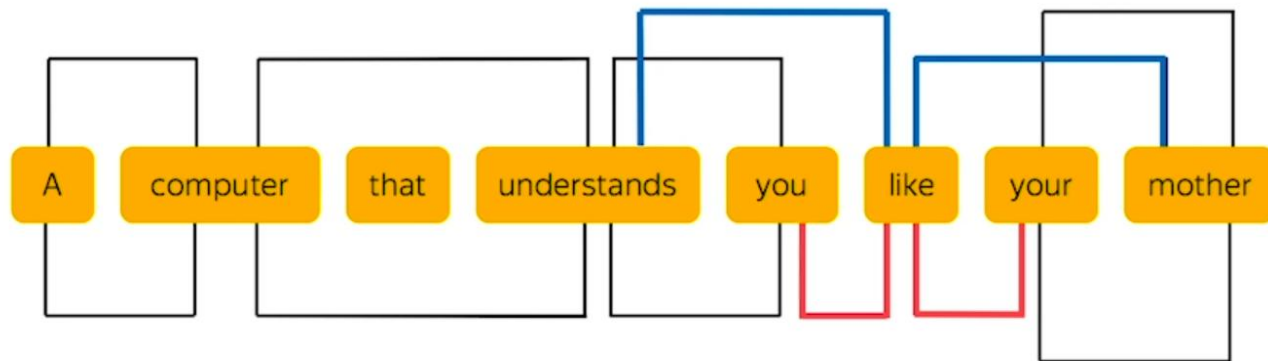
Чего не хватает:

- Multi-head attention
- Positional Encoding

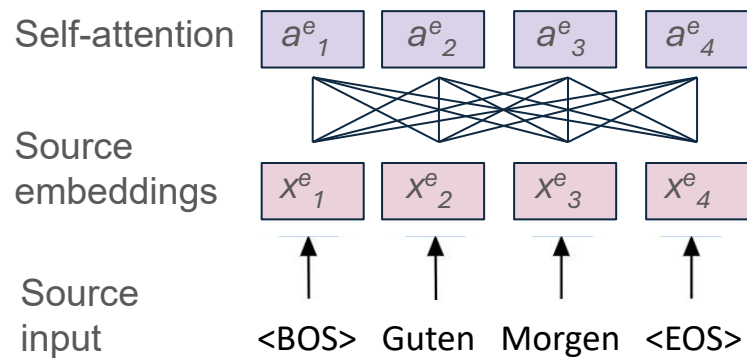


Multi-Head Self-Attention

Между словами в предложении могут быть зависимости разного типа



Multi-Head Self-Attention



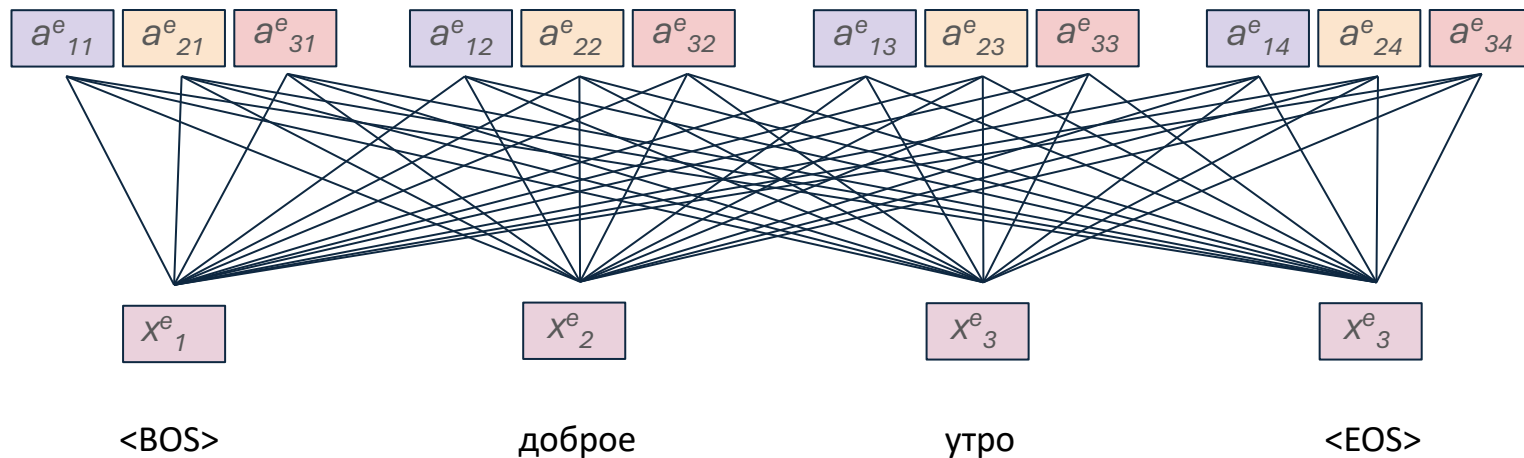
Multi-Head Self-Attention

У всех голов attention
различные веса

k — количество голов
Attention:

$$w_{ij}^k = s^k(x_i^e, x_j^e)$$

$$a_{ki}^e = \sum_{j=1}^4 w_{ij}^k x_j^e$$

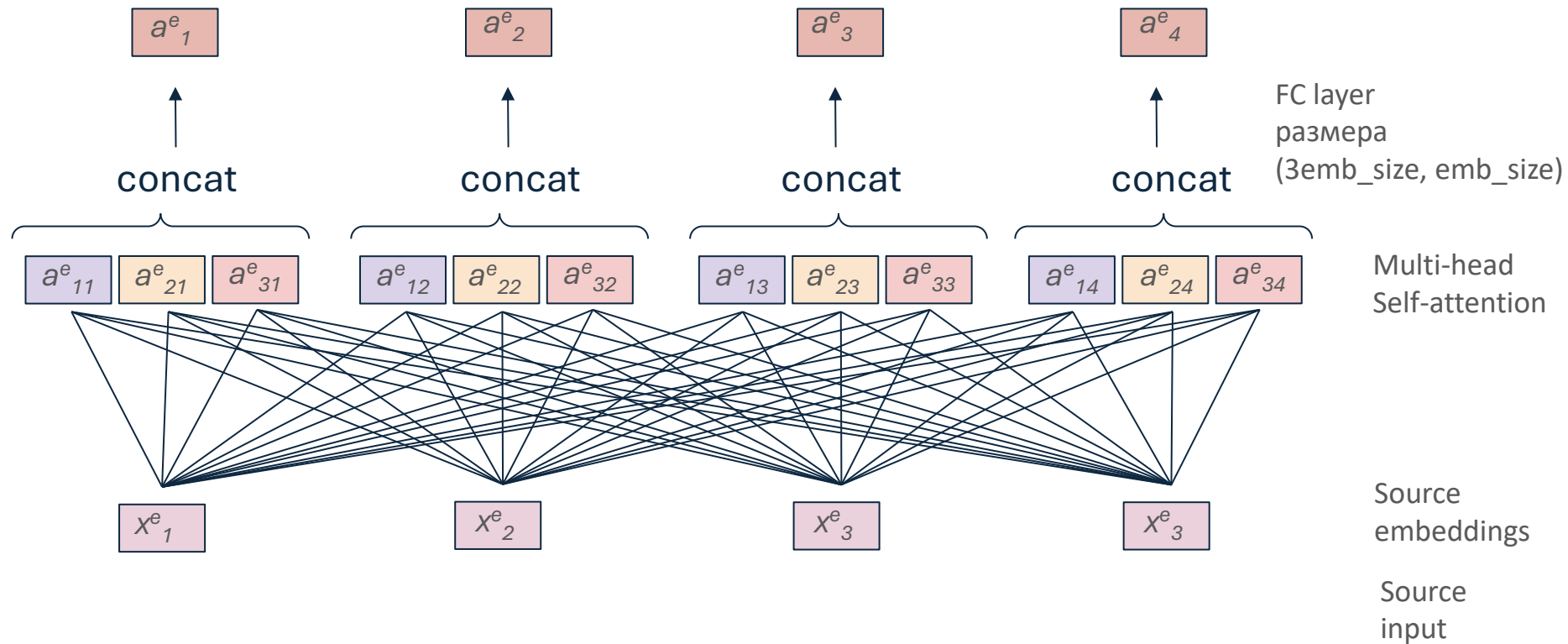


Multi-head
Self-attention

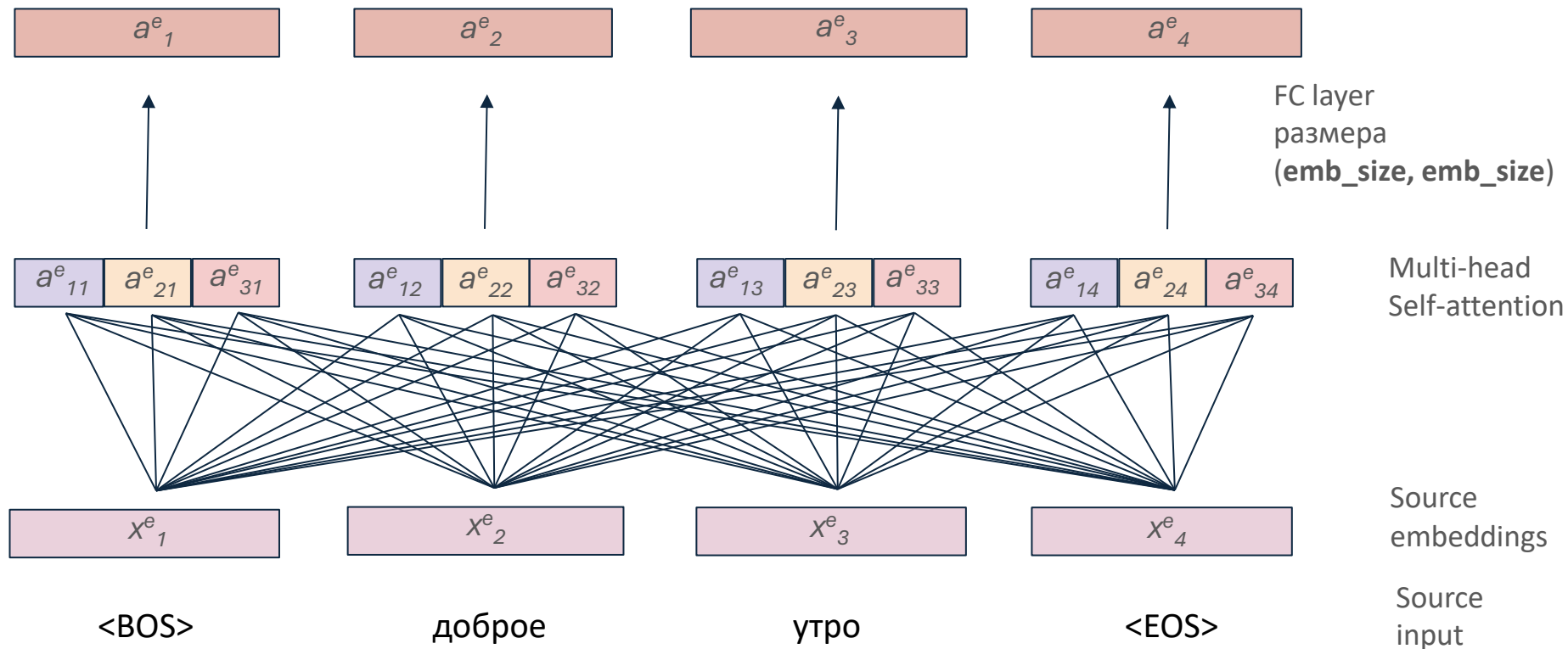
Source
embeddings

Source
input

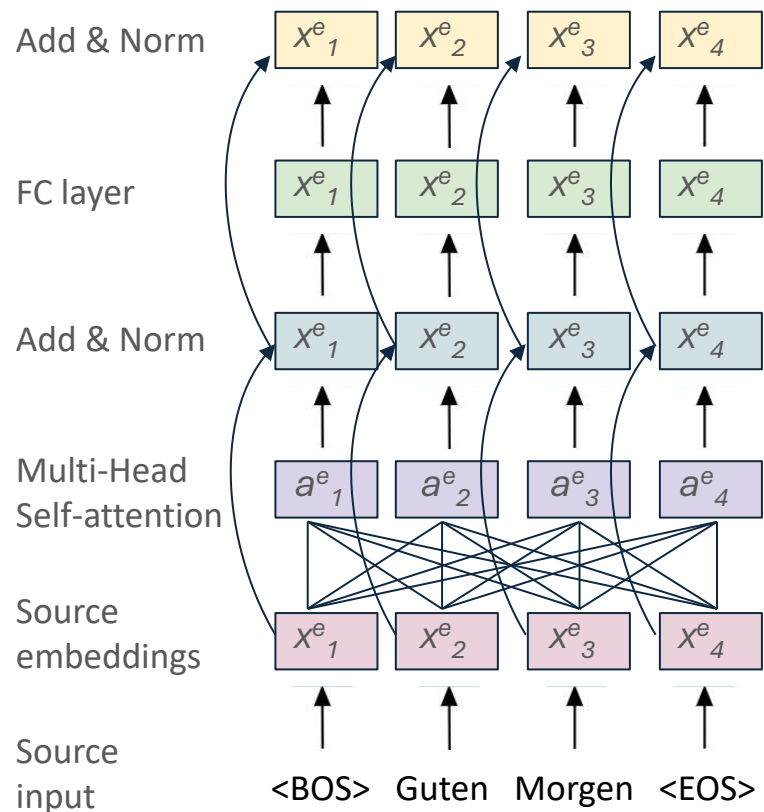
$$a_i^e = \sigma(W[a_{1i}^e, a_{2i}^e, a_{3i}^e] + b)$$



$$a_i^e = \sigma(W[a_{1i}^e, a_{2i}^e, a_{3i}^e] + b)$$

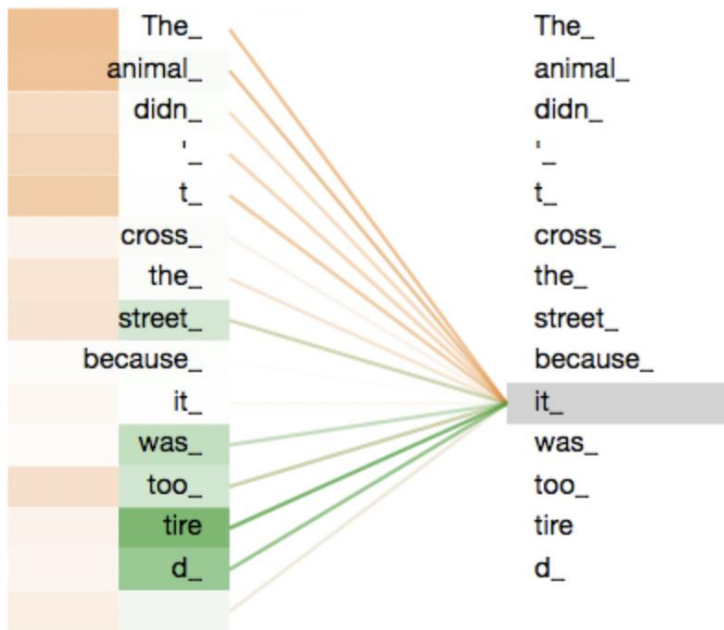


Transformer Encoder



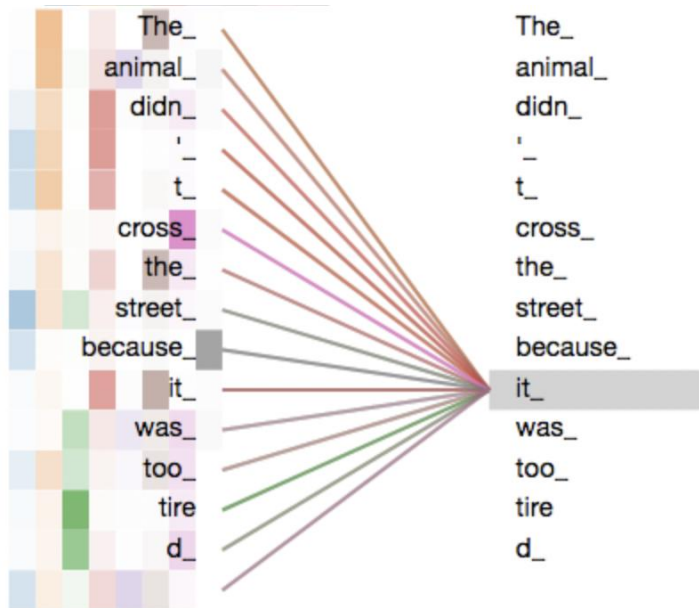
Multi-Head Self-Attention

Разные головы self-attention “обращают внимание”
на разные токены и разную информацию в них



Multi-Head Self-Attention

Разные головы self-attention “обращают внимание”
на разные токены и разную информацию в них

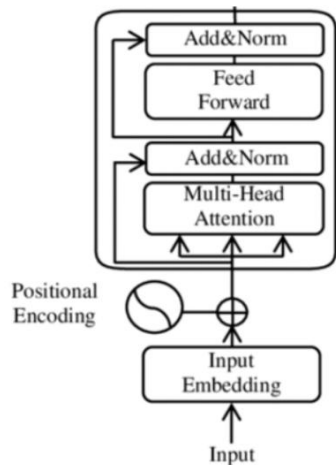


Transformer Encoder

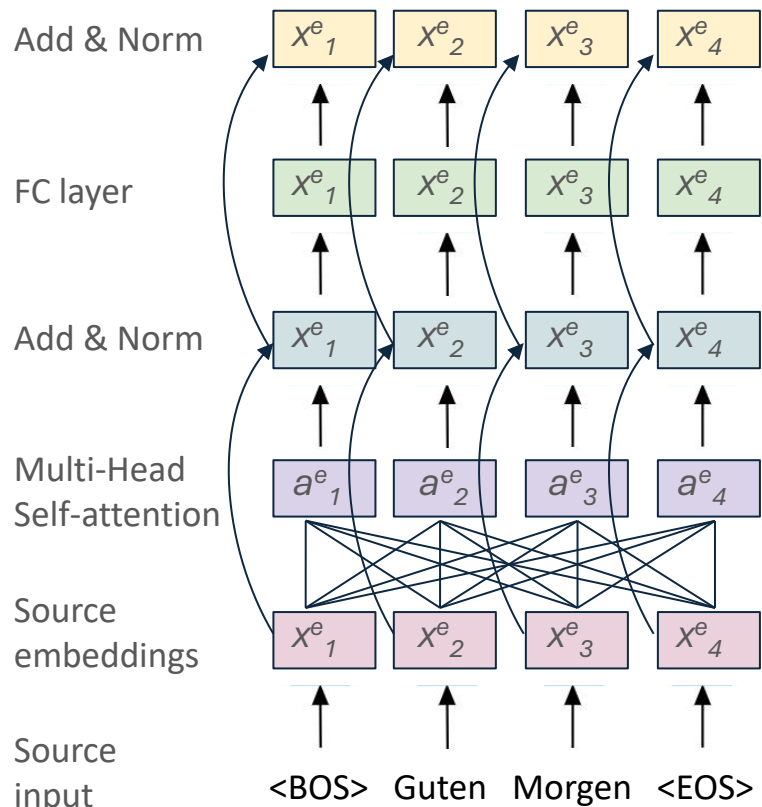
Мы почти собрали Encoder.

Чего не хватает:

- Positional Encoding



Transformer Encoder



Наша сеть не получает информацию о порядке следования токенов в предложении

Но эта информация важна для понимания смысла эмбеддингов и обновления информации о них:

- cat runs horse
- horse runs cat

Positional Encoding

Эмбеддинги с
временным
сигналом



= = = =

Positional
Encodings



+ + + +

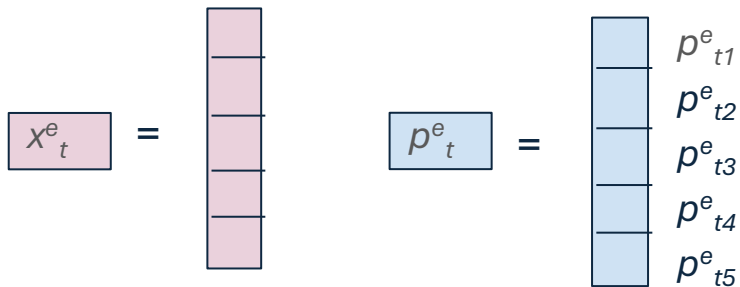
Source
embeddings



Source
input

<BOS> Guten Morgen <EOS>

Positional Encoding



$$p_{ti}^e = \begin{cases} \sin\left(\frac{t}{10000^{\frac{2k}{emb_dim}}}\right), & \text{if } i = 2k \\ \cos\left(\frac{t}{10000^{\frac{2k}{emb_dim}}}\right), & \text{if } i = 2k + 1 \end{cases}$$

Positional Encoding

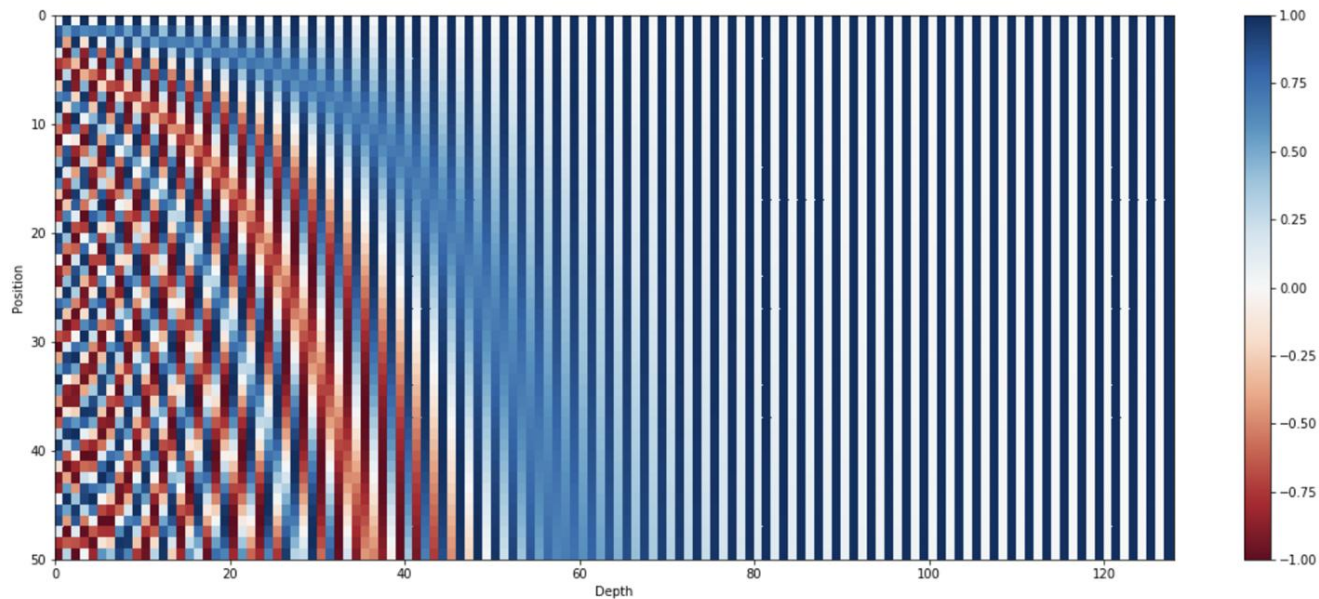


Figure 2 - The 128-dimensional positional encoding for a sentence with the maximum length of 50. Each row represents the embedding vector $\vec{p}_t$

Positional Encoding

Почему функция positional encoding именно такая:

In this work, we use sine and cosine functions of different frequencies:

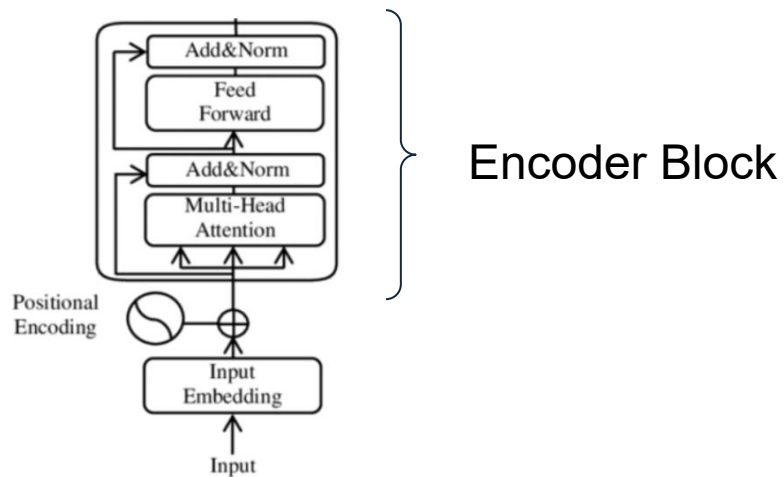
$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{\text{model}}})$$
$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{\text{model}}})$$

where pos is the position and i is the dimension. That is, each dimension of the positional encoding corresponds to a sinusoid. The wavelengths form a geometric progression from 2π to $10000 \cdot 2\pi$. We chose this function because we hypothesized it would allow the model to easily learn to attend by relative positions, since for any fixed offset k , PE_{pos+k} can be represented as a linear function of PE_{pos} .

—

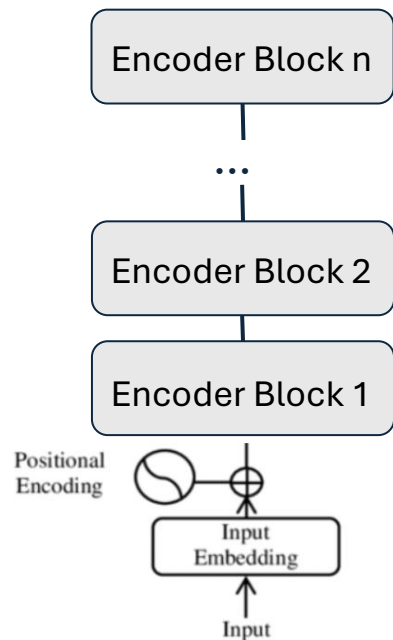
Transformer Encoder

Ура, мы собрали Encoder!

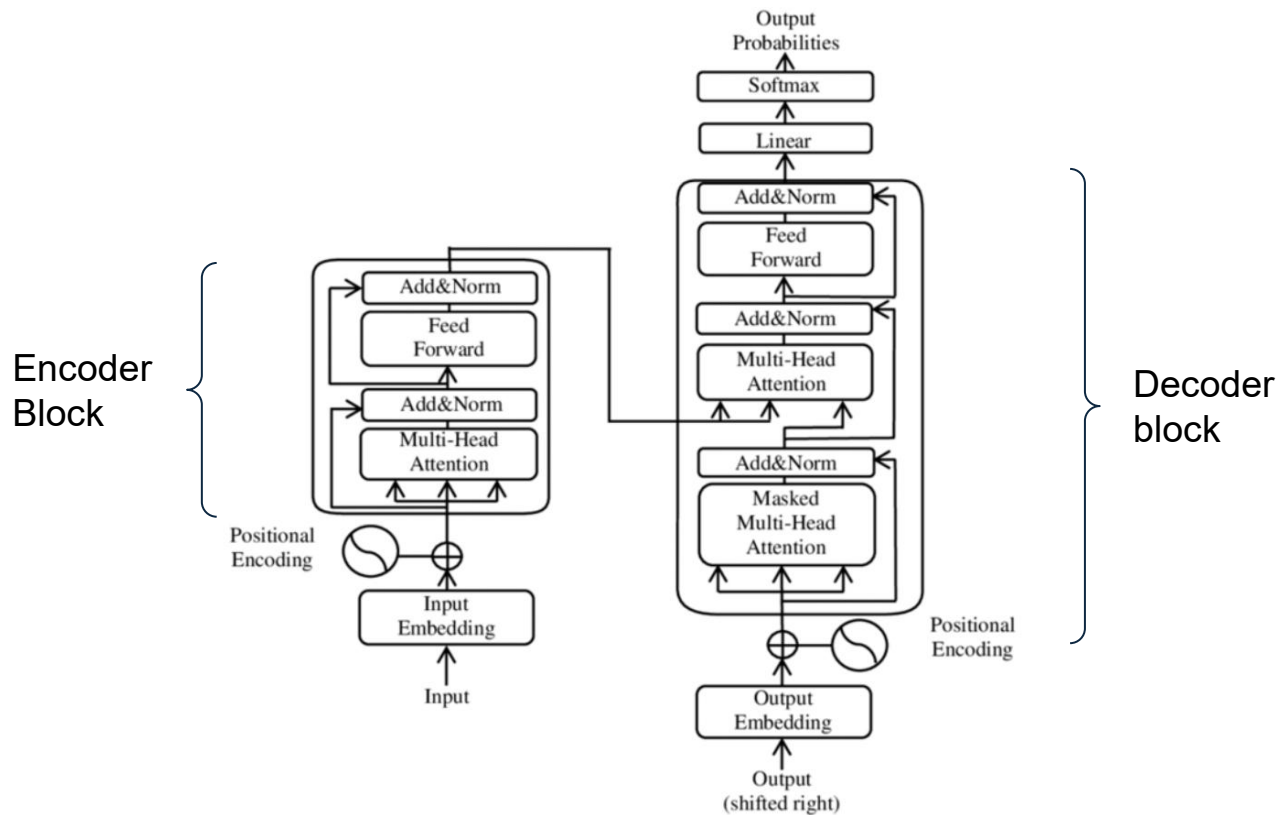


Transformer Encoder

У Encoder могут быть несколько последовательно соединенных блоков



Transformer



Итоги раздела

В этом разделе мы:

- Обсудили идею устройства архитектуры Transformer;
- Детально разобрали устройство encoder блока Transformer;
- Познакомились с такими вещами, как:
 - Self-attention;
 - Multi-head Attention;
 - Positional Encodings

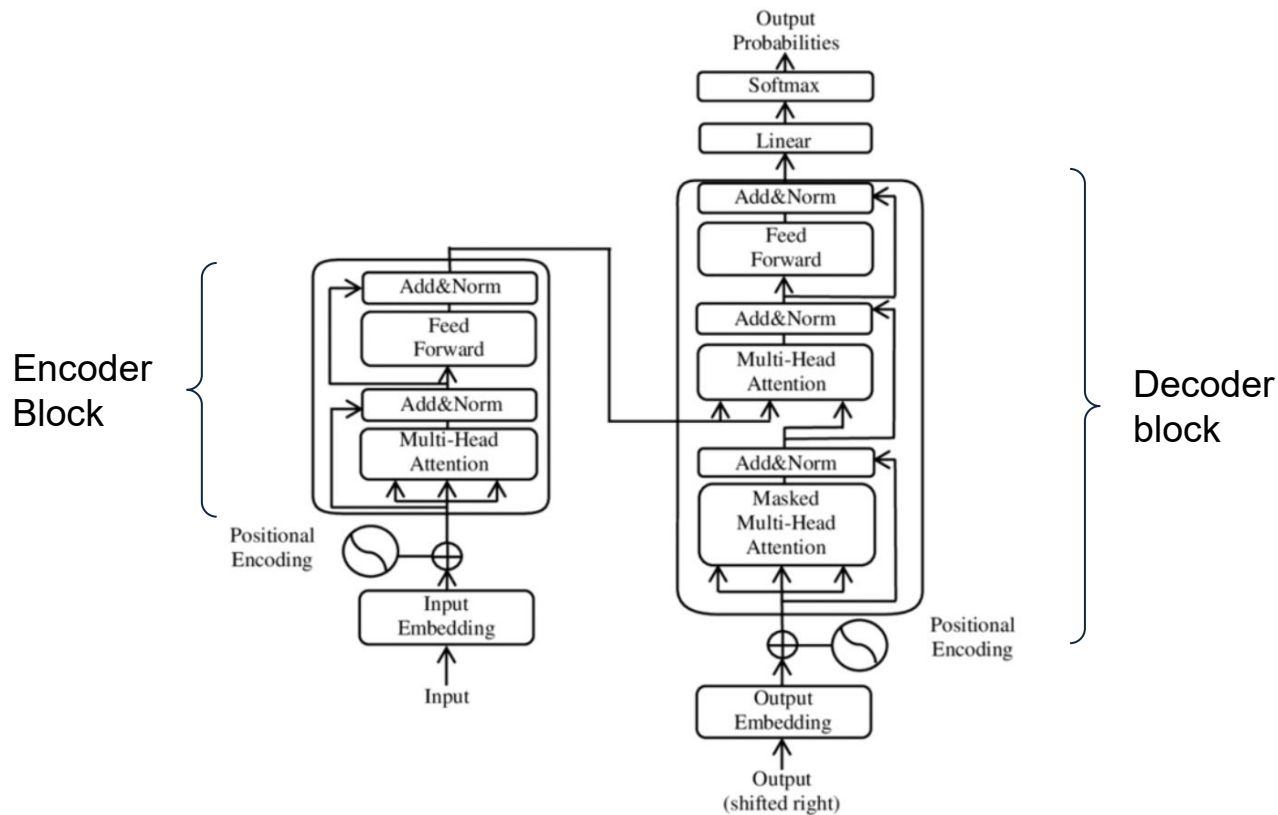
Transformer architecture

Decoder block

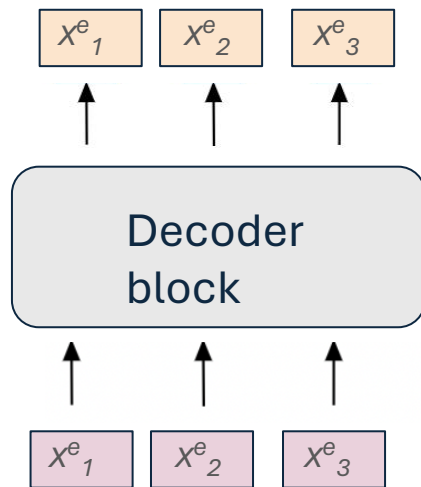
В этом разделе

- Идея архитектуры Transformer
- Устройство Transformer Encoder
- **Устройство Transformer Decoder**
- **QKV Attention**

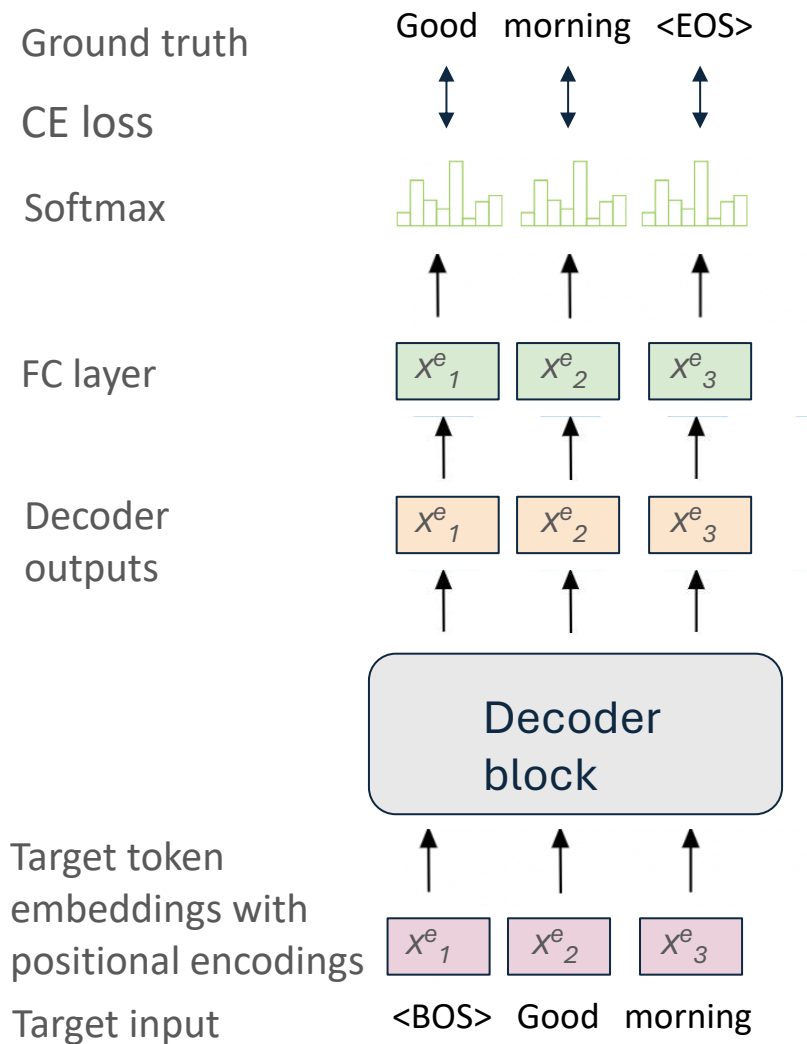
Transformer



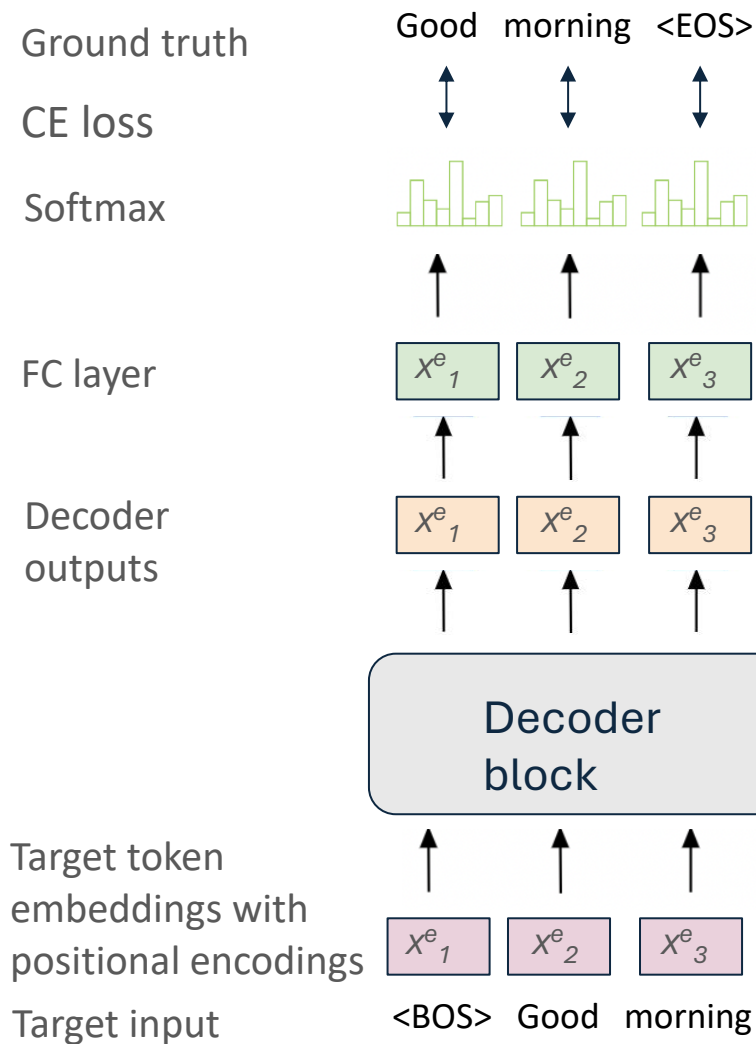
Выходы decoder block
(обновленные
эмбединги)



Decoder block inputs

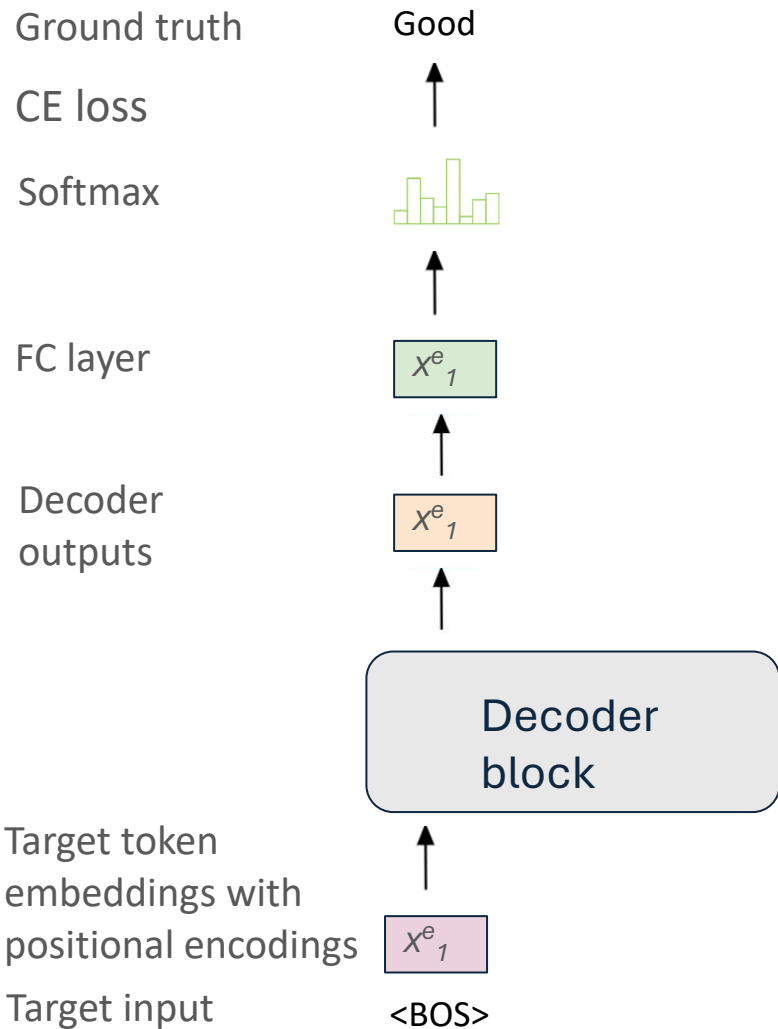


Во время обучения мы можем
подать на вход декодеру сразу
все токены target предложения

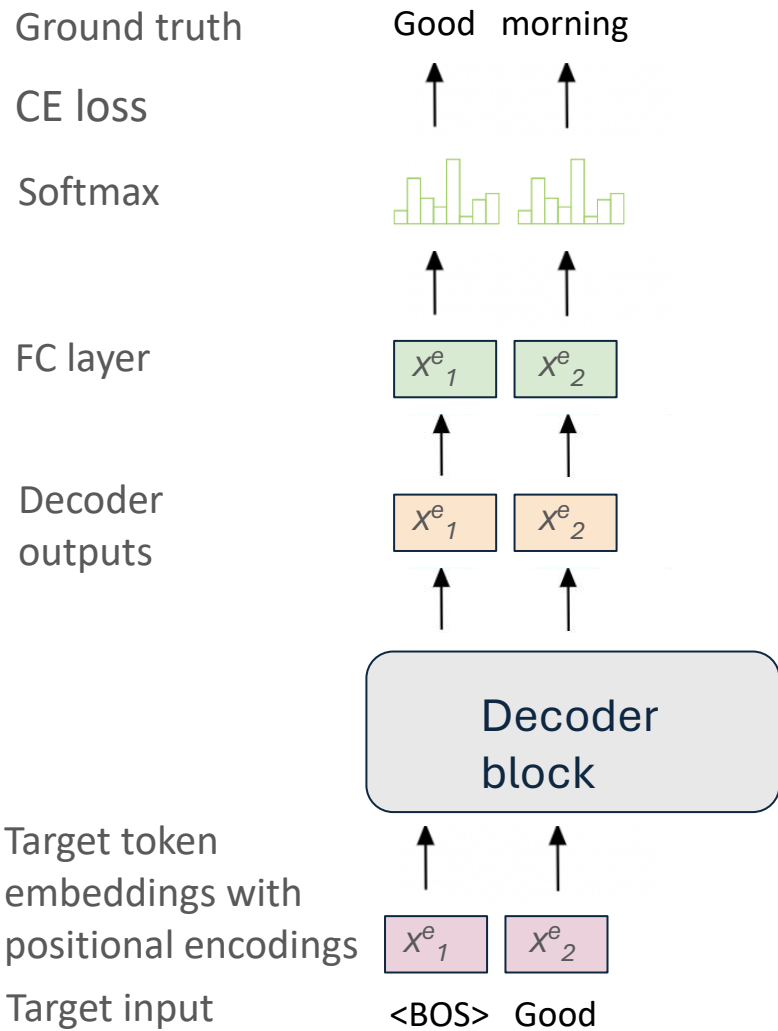


Во время обучения мы можем
подать на вход декодеру сразу
все токены target предложения

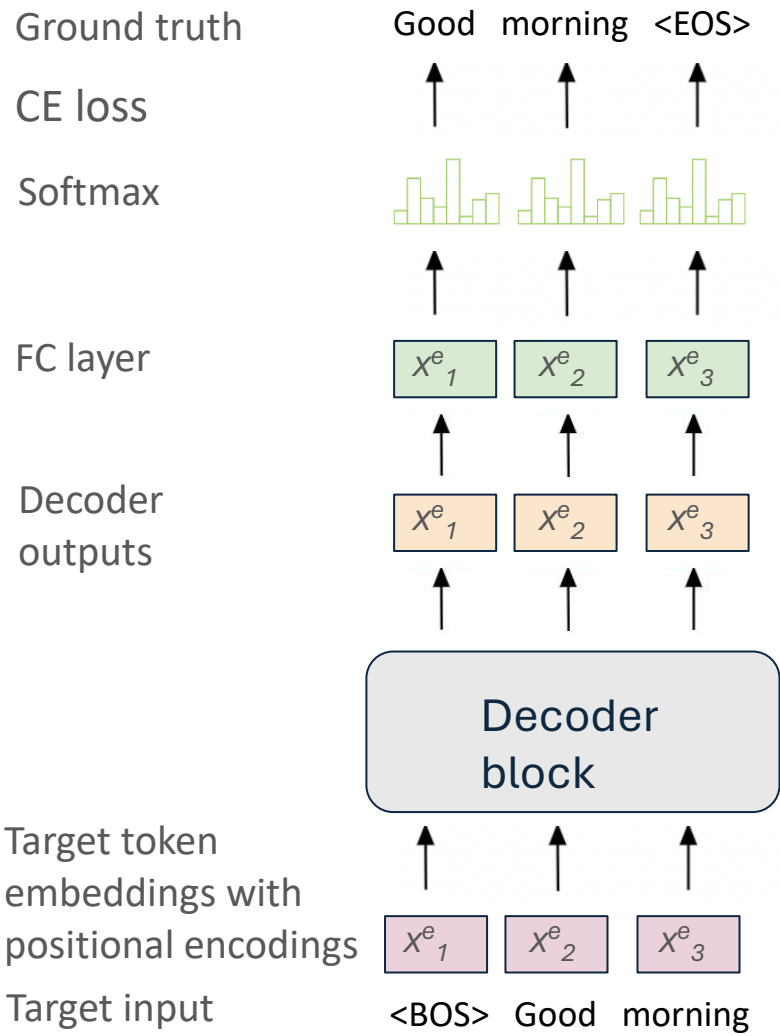
Во время инференса decoder
генерирует выходную
последовательность
авторегрессивно



Во время инференса decoder
генерирует выходную
последовательность
авторегрессивно

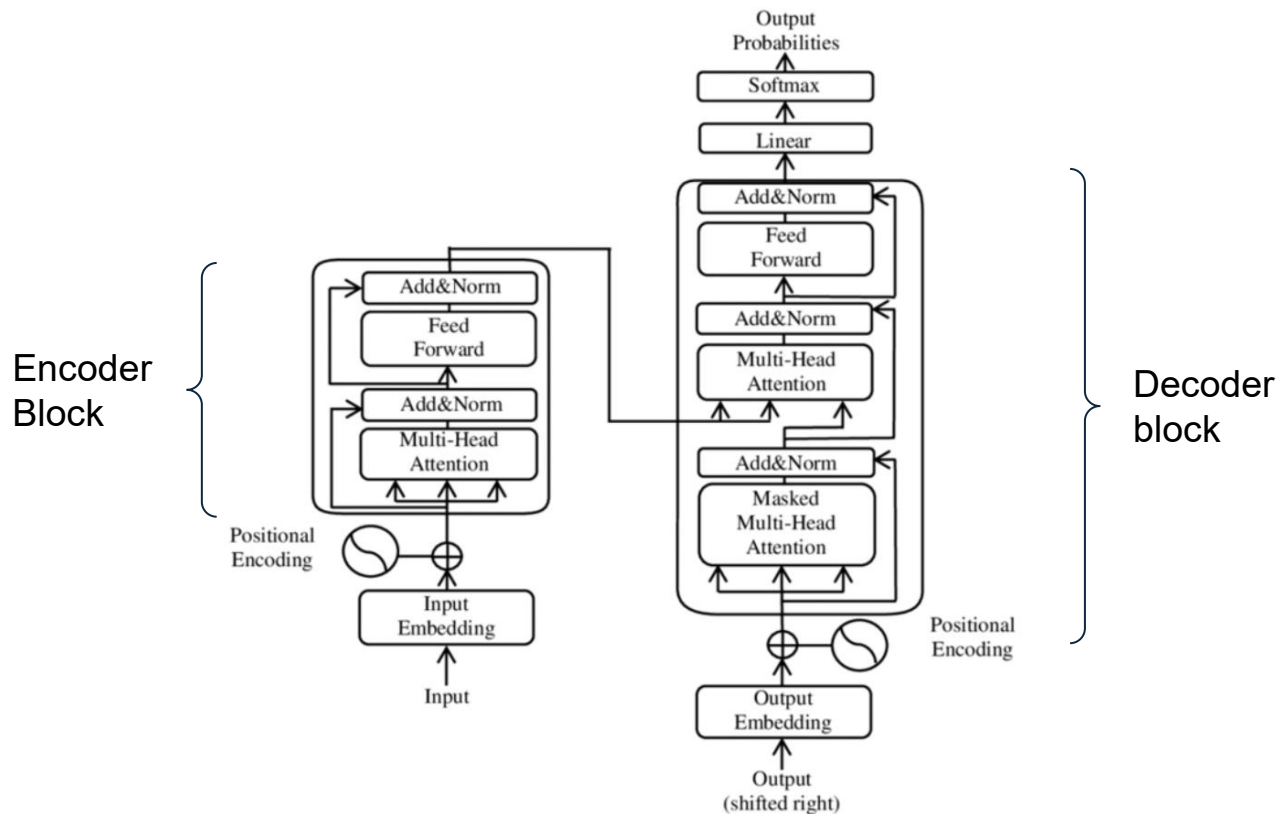


Во время инференса decoder
генерирует выходную
последовательность
авторегрессивно



Во время инференса decoder
генерирует выходную
последовательность
авторегрессивно

Transformer



Masked Self-Attention

Что было в Encoder блоке:

Encoder

Self-attention



Source token embeddings

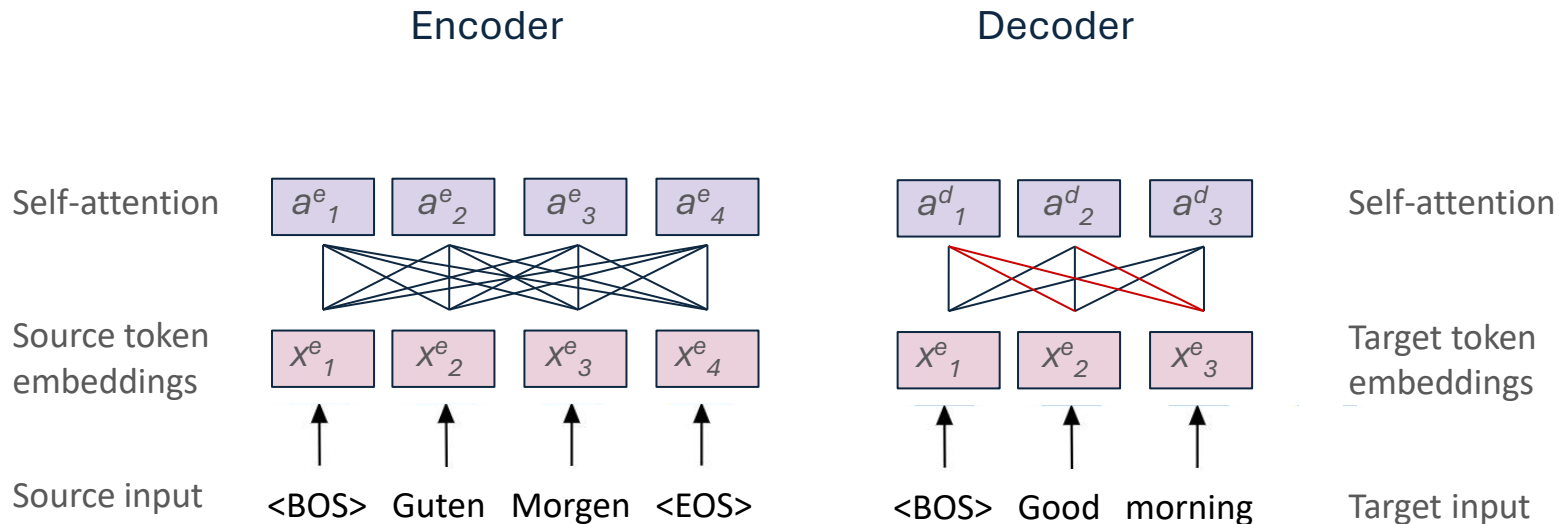


Source input

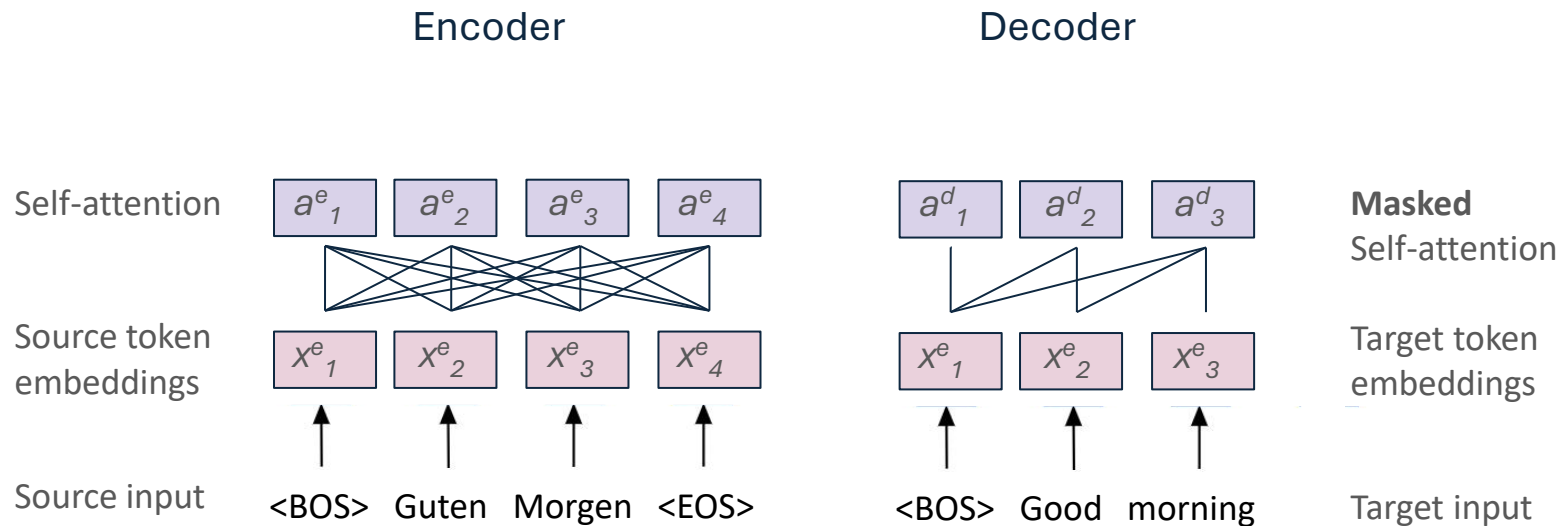
<BOS> Guten Morgen <EOS>

Masked Self-Attention

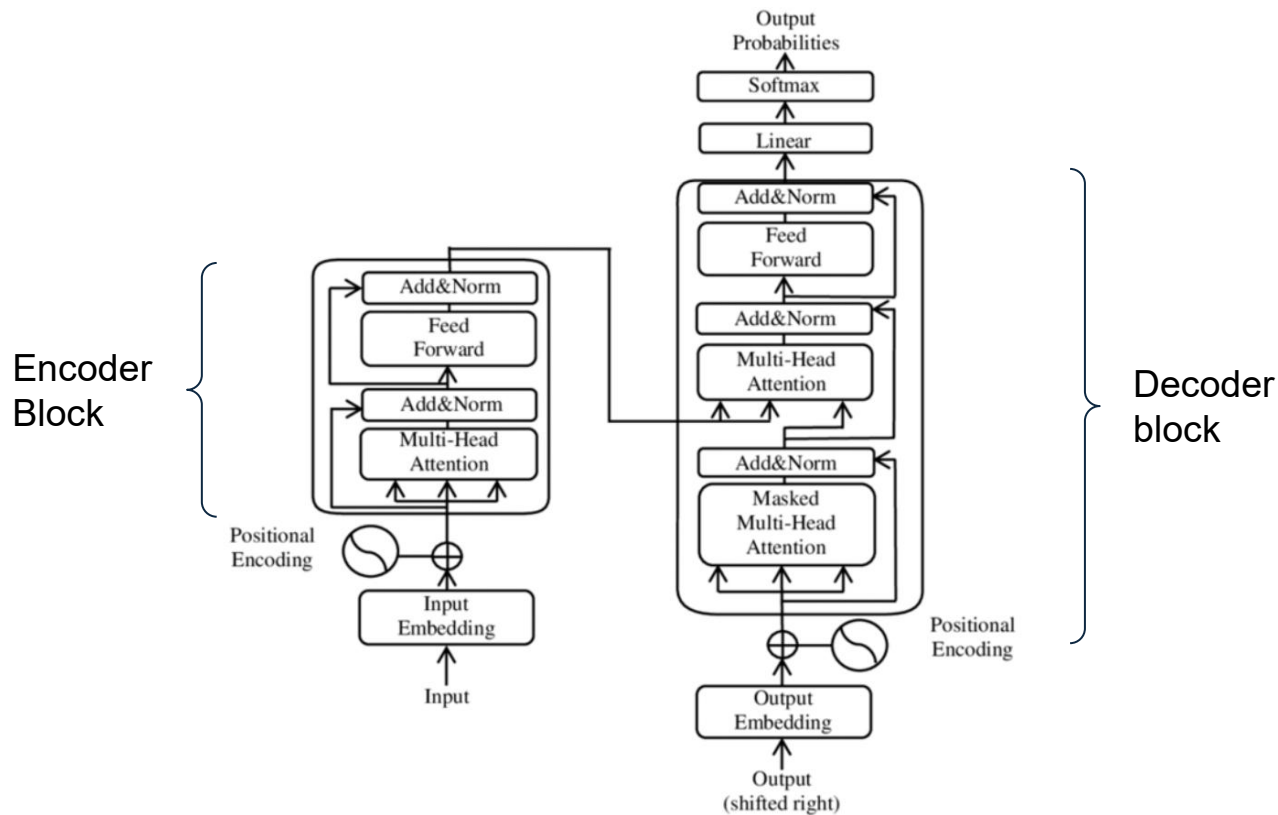
Во время обучения эмбединги x_i^e не могут брать информацию из “будущих” токенов



Masked Self-Attention

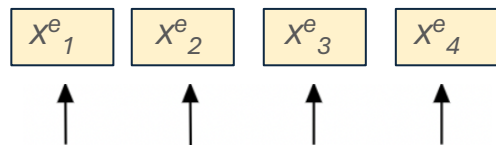


Transformer

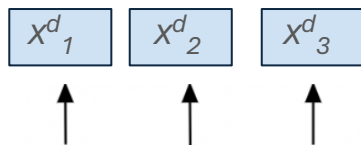


Cross-Attention

Каждый эмбеддинг декодера соберет информацию из эмбеддингов энкодера



Финальные выходы энкодера



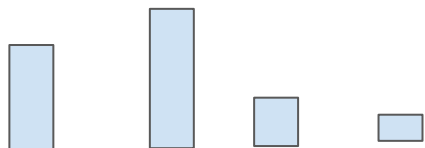
Выходы слоя декодера

$$a_1^{ed} = \sum_{i=1}^4 w_{1i} x_1^d$$

$$a_1^{ed}$$

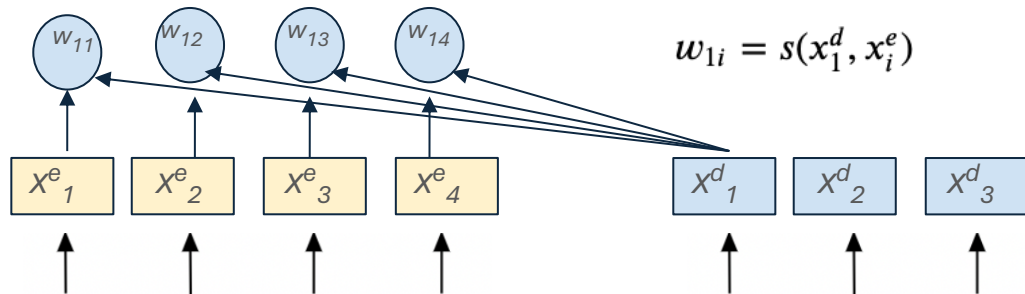


0.4 0.55 0.1 0.05



SoftMax

$$w_{1i} = s(x_1^d, x_i^e)$$



Финальные выходы энкодера

Выходы слоя декодера

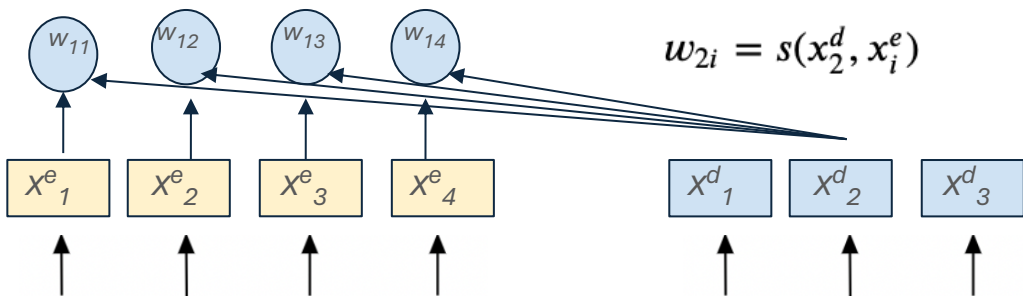
$$a_2^{ed} = \sum_{i=1}^4 w_{2i} x_2^d$$

$$\begin{bmatrix} a_1^{ed} & a_2^{ed} \end{bmatrix}$$

0.1 0.15 0.5 0.25

SoftMax

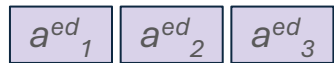
$$w_{2i} = s(x_2^d, x_i^e)$$



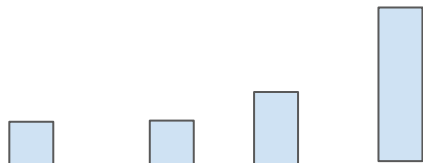
Финальные выходы энкодера

Выходы слоя декодера

$$a_3^{ed} = \sum_{i=1}^4 w_{3i} x_3^d$$

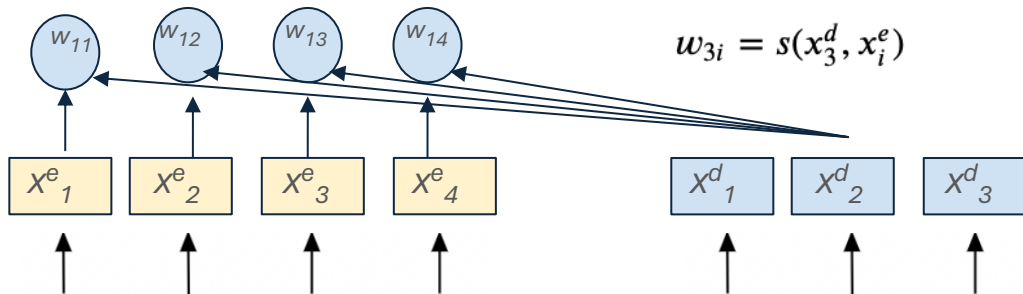


0.11 0.11 0.2 0.56



SoftMax

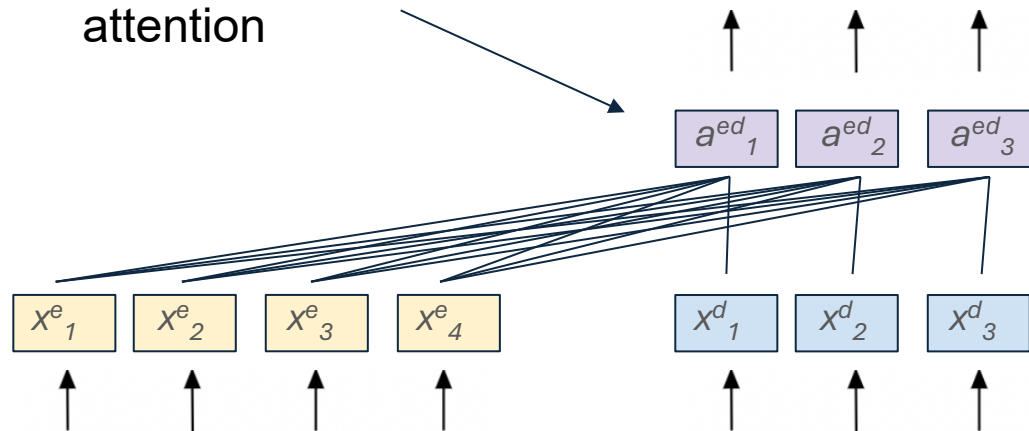
$$w_{3i} = s(x_3^d, x_i^e)$$



Финальные выходы энкодера

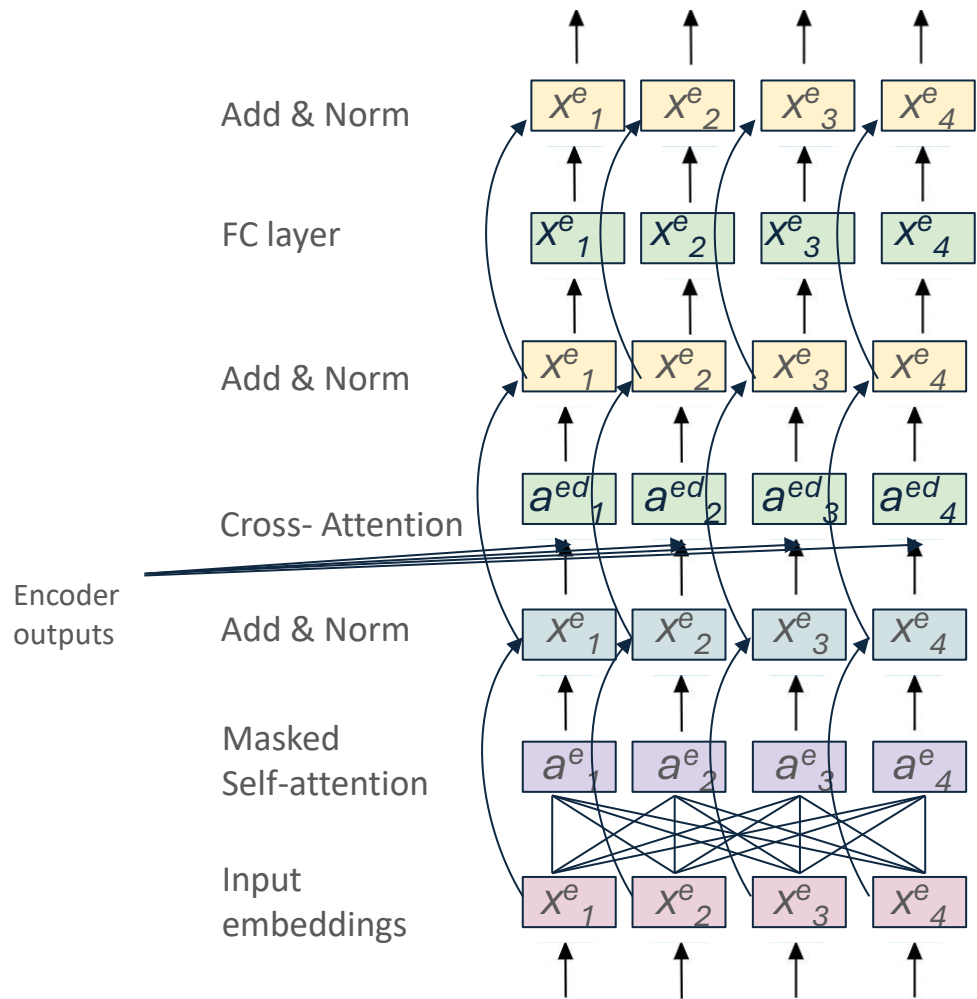
Выходы слоя декодера

Векторы cross-attention

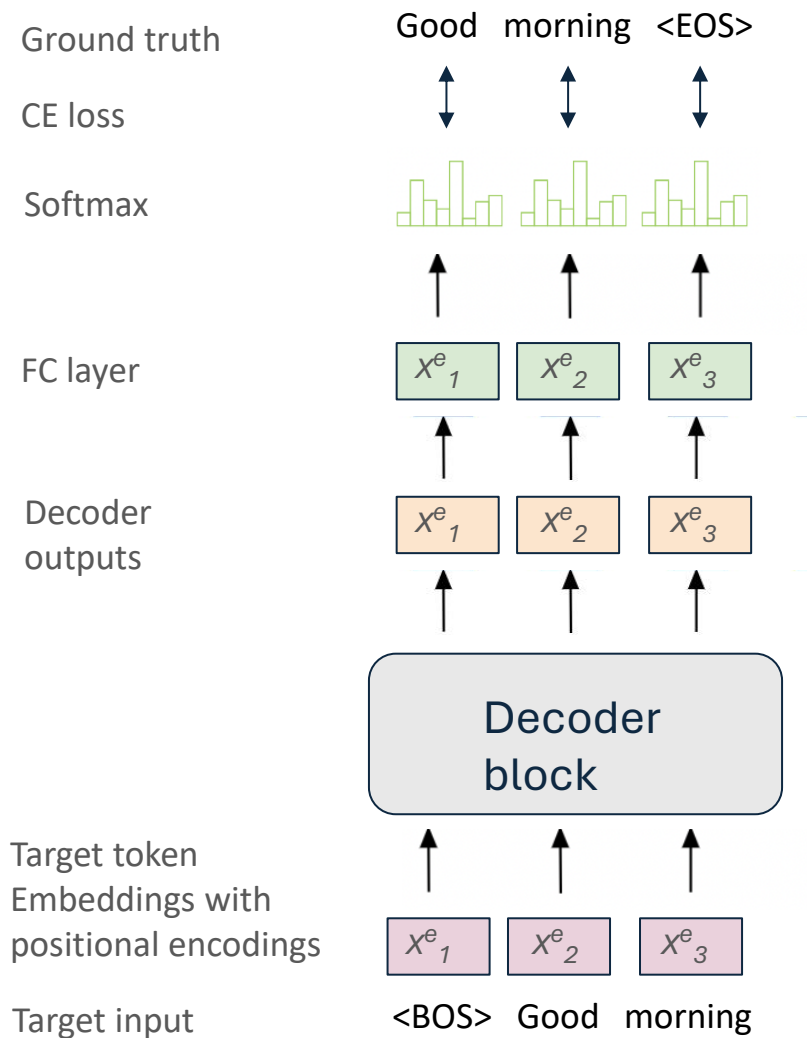


Финальные выходы энкодера

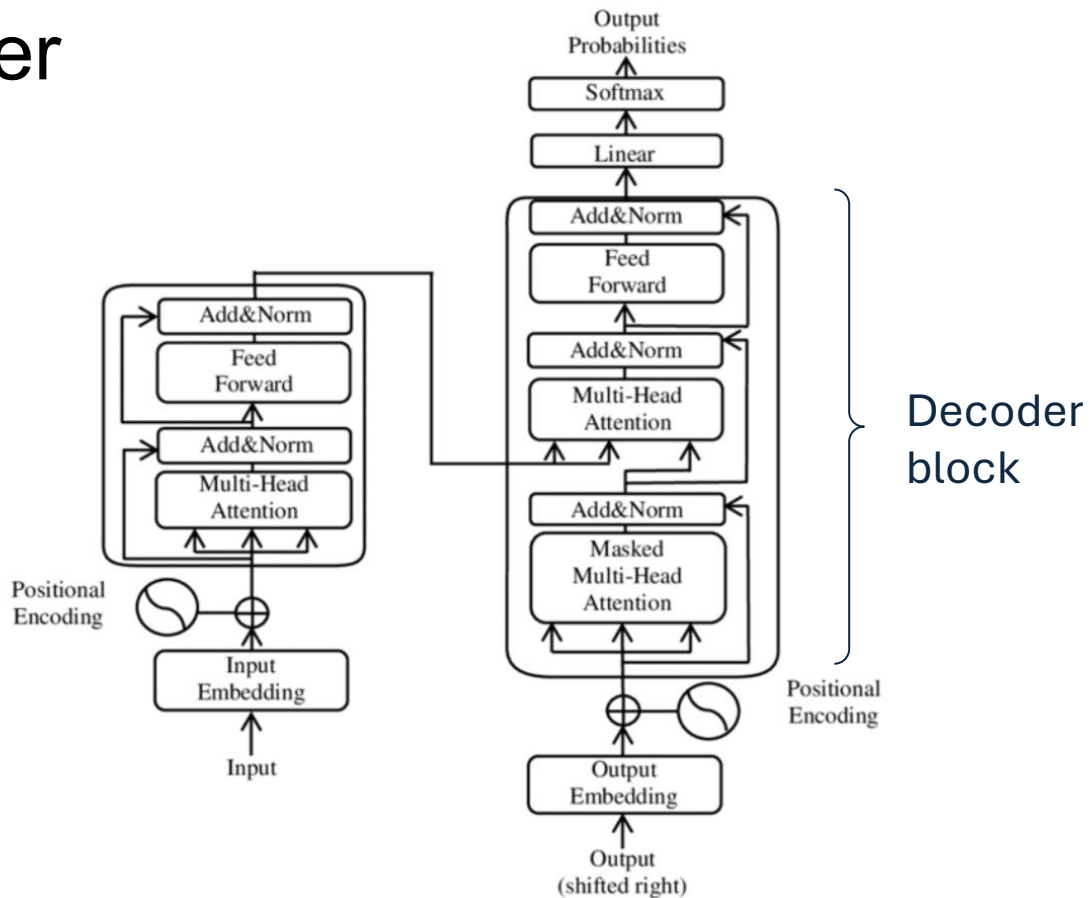
Выходы слоя декодера



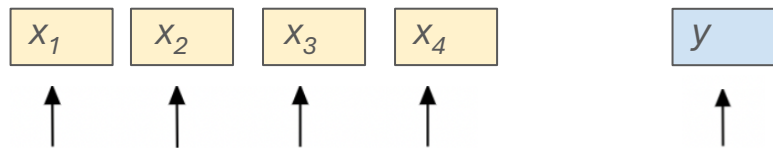
Decoder block



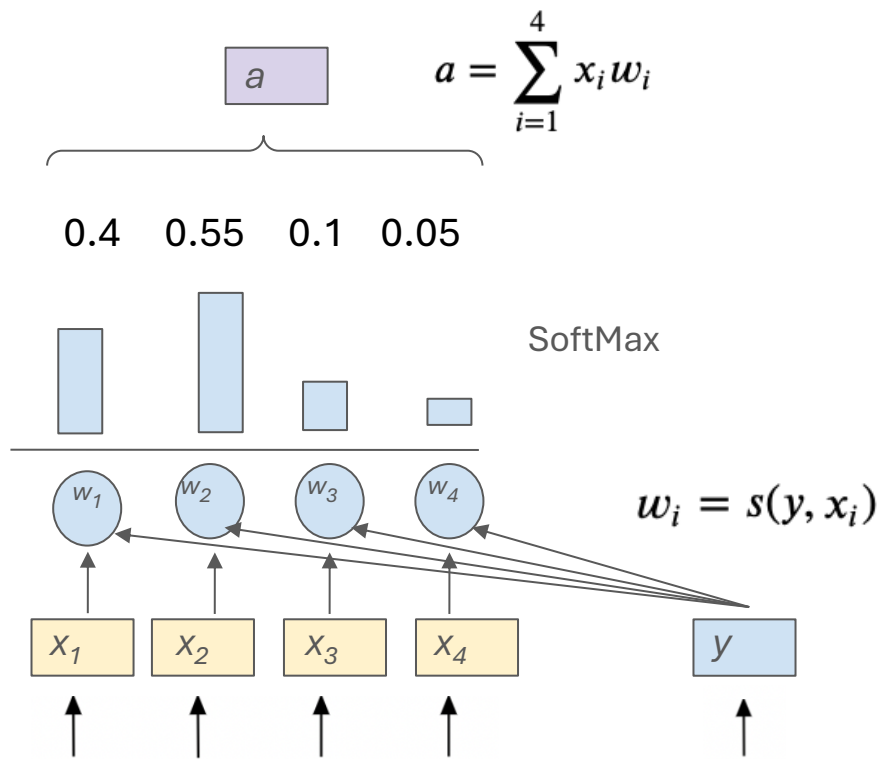
Transformer Decoder



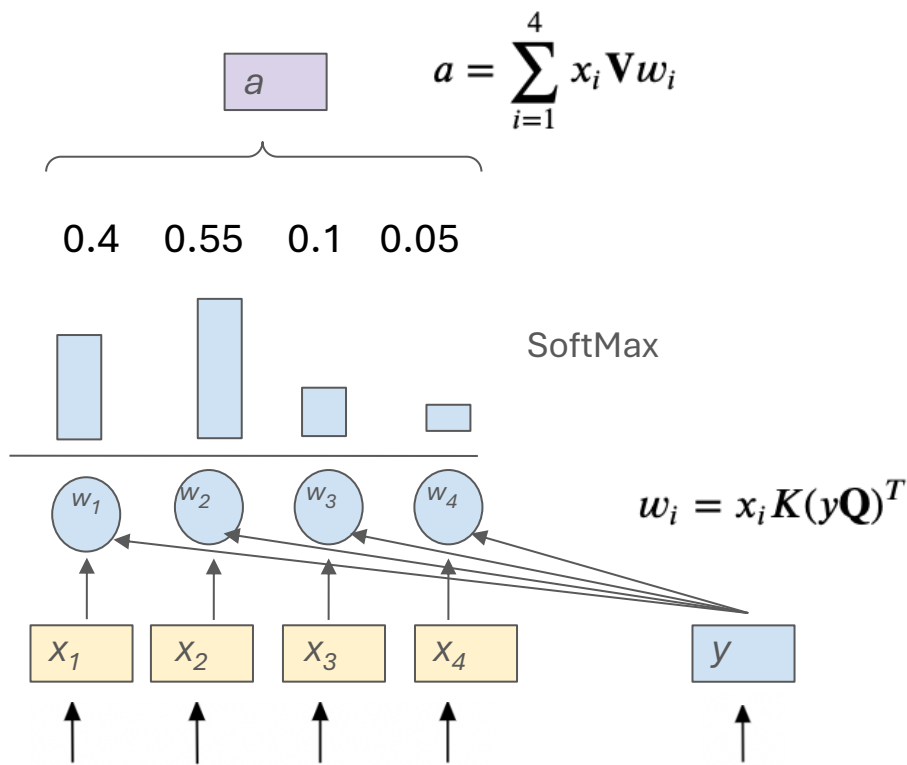
QKV Attention



QKV Attention



QKV Attention



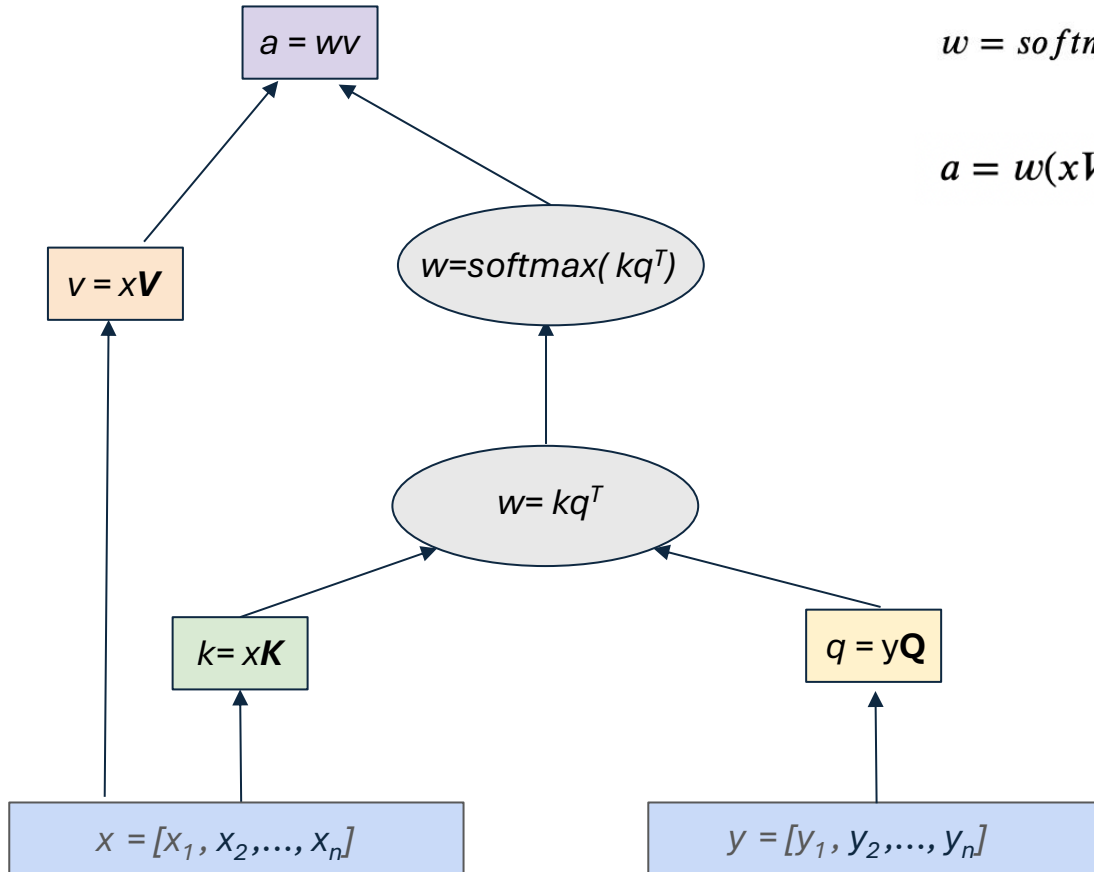
У QKV Attention 3 обучаемые матрицы::

- Q_{dxm} (query)
- K_{dxm} (key)
- V_{dxm} (value)

Здесь:

- D — длина эмбеддингов;
- M можно выбирать любым

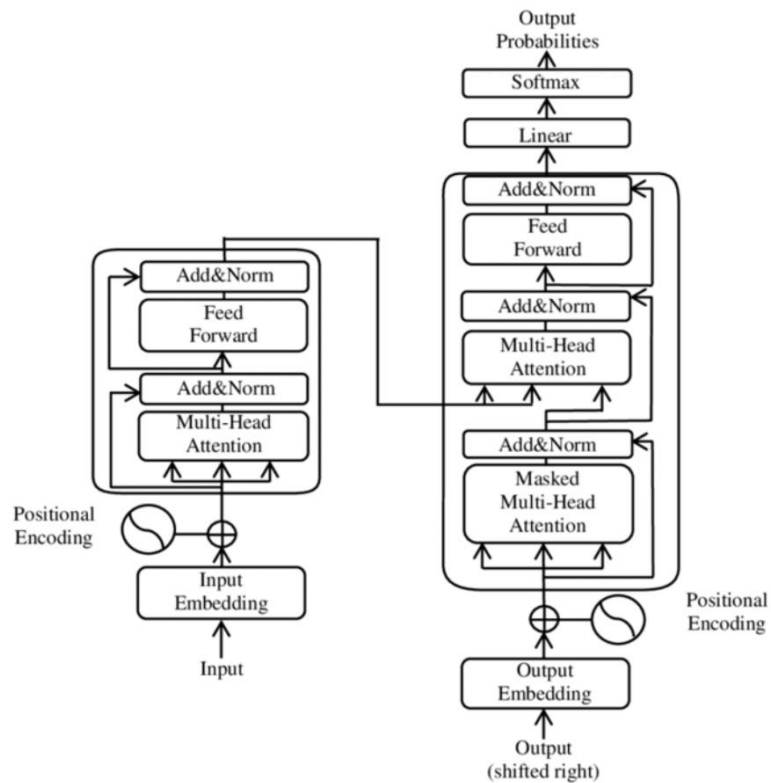
QKV Attention в векторной форме:



$$w = \text{softmax} \left(\frac{xK(yQ)^T}{\sqrt{m}} \right)$$

$$a = w(xV)$$

Transformer



Итоги раздела

В этом разделе мы:

- Детально разобрали устройство decoder блока Transformer;
- Собрали полную архитектуру Transformer;
- Узнали, как работает QKV Attention.

Итоги раздела

Transformer blocks structures:

- LayerNorm;
- Self-Attention
- Masked Self-Attention
- Cross-Attention
- QKV Attention

Использованные материалы

Видеолекции Deep Learning School

- [Рекуррентная нейронная сеть](#)
- [Машинный перевод, архитектура Seq2Seq](#)
- [Механизм внимания \(Attention\)](#)
- [Архитектура Transformer. Введение, Transformer Encoder](#)
- [Архитектура Transformer. Decoder, QKV Attention](#)